SISTEMAS DIGITAIS II - 2026/27

TECNOLOGIA SETÚBAL
ESCOLA SUPERIOR POLITÉCNICO SETÚBAL

Laboratório Nº	2
Data	28/10/2026
Docente	José Sousa
Curso	LEEC
Turma	PL1

Alunos		
	202300553	Rodrigo Silva Costa
	2025184378	Diego Alejandro Castro Laya

2.º Trabalho de Laboratório

Gestão dos displays para o jogo Guitar Hero

(Realizações Diretas)

Índice

1 Introdução....2
2 Especificação e arquitetura do sistema....4
2.1 Especificação funcional....4
2.2 Arquitetura a controlar e controlador....4
2.3 Sinais de interface e a sua natureza....5
3 Sincronização das teclas....7
3.1 Sincronizador e anti-ressalto da tecla Start....7
3.2 Sincronizador do Reset....8
4 LFSR de 32 bits....10
5 Memória 4x2....12
6 Temporizador variável....14
7 LED Beat....17
8 Gestão dos displays....18
9 Controlador: máquina de estados por realização direta....21
9.1 Algoritmo e fluxograma....21
9.2 Tabela de estados primitiva (passo 1)....22
9.3 Redução de estados (passos 2 a 4)....23
9.4 Codificação dos estados (passo 5)....24
9.5 Tabela de transições (passo 6)....24
9.6 Saídas nas transições (passo 7)....25
9.7 Equações de excitação e de saída (passos 8 e 9)....25
9.8 Diagrama lógico e código Verilog estrutural (passo 10)....26
9.9 Verificação do controlador (passo 11)....29
10 Integração do sistema....30
10.1 Circuito de topo....30
10.2 Restrições e implementação....32
10.3 Simulação do sistema completo....33
10.4 Integração faseada e ensaio na placa....34
11 Análise crítica do código de fontes externas....36
11.1 LFSR do enunciado....36
11.2 Módulos do 1.º trabalho....36
11.3 Primitivas FDPE e FDCE....36
11.4 IA generativa....36
12 Conclusão....37
Anexo A Testbenches....38

1 Introdução

Este relatório descreve o 2.º Trabalho de Laboratório de Sistemas Digitais II: um sistema para a gestão dos *displays* de 7 segmentos do jogo Guitar Hero na placa Basys3. Um LFSR de 32 *bits* gera uma sequência pseudo-aleatória de cordas e pausas, uma memória de 4x2 *bits* guarda as cordas mostradas nos quatro *displays* e um temporizador variável define o ritmo a que a sequência avança (e acelera) ao longo do "braço da guitarra".

O sistema está dividido em **arquitetura a controlar** e **controlador**. O controlador é uma máquina de estados síncrona por **realização direta** (um *flip-flop* por estado), desenhada a partir do fluxograma do algoritmo segundo a metodologia lecionada e descrita em Verilog no **modelo estrutural**, de forma que cada *flip-flop* corresponde a um estado do fluxograma.

A secção 2 apresenta a especificação, a arquitetura e os sinais de interface; as secções 3 a 8 descrevem os módulos da arquitetura a controlar (código, *testbench* e simulação); a secção 9 descreve o projeto do controlador, pelos 11 passos da metodologia; a secção 10 trata da integração, das restrições, da implementação e do ensaio na placa; a secção 11 analisa o código de fontes externas e a secção 12 conclui. As listagens dos *testbenches* estão no Anexo A.

Ferramentas: Vivado 2026.1 (licença WebPACK) para a FPGA `xc7a35ticpg236-1L` da Basys3 e draw.io para os diagramas. Todos os *testbenches* aplicam uma sequência curta de entradas e conferem as saídas com os valores esperados, escritos no próprio *testbench*, terminando com `Simulacao OK` ou `Simulacao FALHOU`; os tempos longos (varrimento, anti-ressalto e temporizador) são parâmetros dos módulos e são reduzidos nas simulações.

2 Especificação e arquitetura do sistema

2.1 Especificação funcional

O sistema tem duas teclas, **Reset** (BTNU) e **Start** (BTNC), os quatro *displays* do jogo e os LEDs de sinalização **Ready** (LD0), **Beat** (LD1) e **Waiting** (LD2). Depois de Reset (secção 2.1 do enunciado):

- inicializa a zeros a memória da sequência (os quatro *displays* ficam apagados) e habilita o LFSR, que passa a avançar livremente;
- acende o LED Ready e espera que se carregue e largue a tecla Start;
- quando se larga Start, apaga o LED Ready e, em ciclo: desloca a memória uma posição, carregando 2 *bits* do LFSR na primeira posição e perdendo a última; troca o estado do LED Beat; inicializa o temporizador variável e espera pelo fim do tempo; acelera ligeiramente o temporizador (parte opcional) e repete.

A tecla Reset tem uma única função: pôr a máquina de estados no estado inicial, de forma **assíncrona** (secção 2.7 do enunciado). Os restantes elementos são inicializados pelas saídas da própria máquina de estados no estado inicial. As únicas exceções são o sincronizador da tecla Start e o varrimento dos *displays* (divisor de *clock* e contador): o primeiro só condiciona uma entrada e o varrimento funciona sempre, arrancando com os valores iniciais da configuração da FPGA; nenhum deles guarda informação do jogo.

O enunciado não define a função do LED Waiting. Como o LED Ready já indica a espera pela tecla Start, o Waiting indica a espera pelo fim do tempo de cada passo (estado S4): fica aceso quase sempre durante o jogo e apaga-se por instantes em cada passo. Para depuração, os LEDs LD10 a LD15 mostram os seis *flip-flops* da máquina de estados (um aceso de cada vez).

2.2 Arquitetura a controlar e controlador

A Figura 1 mostra o diagrama de blocos do sistema e a Tabela 1 a função de cada bloco. As saídas do controlador são os comandos sobre a arquitetura (ClrMem, InitLFSR, EnLFSR, ShiftMem, RstTemp, StartTemp, Faster e TglBeat); as saídas da arquitetura Start e Tempo são as entradas do controlador, fechando a malha de controlo (Fig. 2 do enunciado).

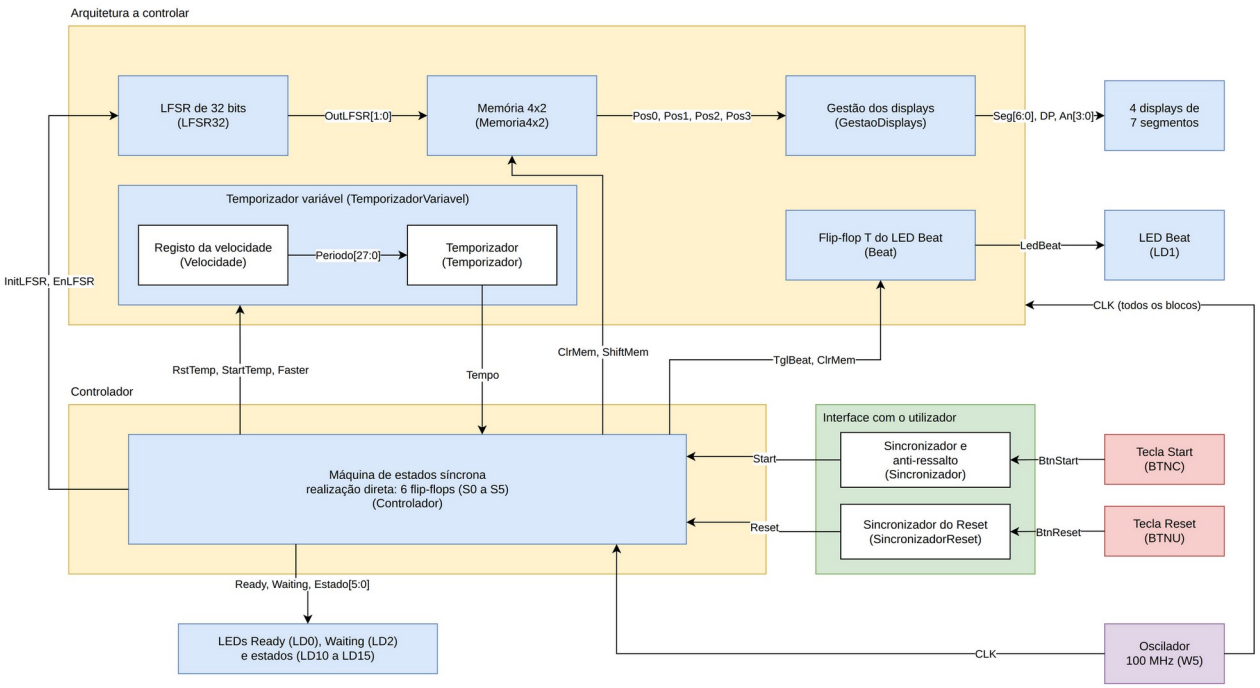


Figura 1: Diagrama de blocos do sistema: arquitetura a controlar, controlador e interface com o utilizador.

Tabela 1: Blocos do sistema, módulos Verilog e origem.

Bloco	Módulo (ficheiro)	Função	Origem
LFSR de 32 *bits*	LFSR32 (lfsr32/lfsr32.v)	Sequência pseudo-aleatória das cordas (2 *bits* por passo)	Enunciado (secção 2.2)
Memória 4x2	Memoria4x2 (memoria4x2/)	Cordas dos 4 *displays*, organizadas em fila	4 × Registo4 do 1.º trabalho
Gestão dos *displays*	GestaoDisplays, TranscodCordas (gestaodisplays/)	Varrimento dos 4 *displays* e transcodificação das cordas	Sistema do 1.º trabalho, transcodificador alterado
Temporizador variável	Velocidade, Temporizador, TemporizadorVariavel (temporizador/)	Tempo de cada passo e aceleração do jogo	Novo
LED Beat	Beat (beat/beat.v)	*Flip-flop* T: o LED troca de estado em cada passo	Novo (primitiva FDCE)
Sincronizador e anti-ressalto	Sincronizador (sincronizador/)	Tecla Start sincronizada com o relógio e sem ressaltos	Novo
Sincronizador do Reset	SincronizadorReset (sincronizador/)	Reset ativado de imediato e libertado em sincronismo com o relógio	Novo (primitivas FDPE)
Controlador	Controlador (controlador/controlador.v)	Máquina de estados por realização direta (6 *flip-flops*)	Novo (FDPE, FDCE e portas lógicas)
Circuito de topo	GuitarHero (guitarhero/)	Liga o controlador à arquitetura e aos pinos da placa	Novo

Para além dos blocos sugeridos no enunciado (LFSR, memória e temporizador), a arquitetura inclui os seguintes:

- **dois sincronizadores**, porque as teclas mudam em qualquer instante: a tecla Start é testada nas decisões da máquina de estados e tem de chegar síncrona e sem ressaltos (secção 3.1); a libertação do Reset tem de ser síncrona para os seis *flip-flops* saírem de S0 no mesmo flanco (secção 3.2);
- um **registo da velocidade**, que guarda o período do temporizador e o encurta em cada passo (parte opcional);
- um ***flip-flop* T** para o LED Beat, que troca de estado com um único comando (TglBeat) por passo.

Os LEDs Ready e Waiting dispensam o *latch* SR sugerido na secção 2.4 do enunciado: as saídas que os comandam estão ativas em todos os estados entre o momento em que o LED acende e aquele em que apaga (Ready em S1 e S2, Waiting em S4). Todos os blocos usam o relógio de 100 MHz da placa.

2.3 Sinais de interface e a sua natureza

A Tabela 2 identifica os sinais entre o controlador, a arquitetura e a placa. Todas as saídas do controlador são de Moore (só dependem do estado) e ativas a *High*.

Tabela 2: Sinais de interface do controlador e da arquitetura a controlar.

Sinal	Bits	Origem → destino	Natureza	Função
CLKBasys	1	Oscilador (W5) → todos os blocos	Relógio, flanco ascendente	Relógio único de 100 MHz
BtnReset	1	Tecla BTNU → Sincronizador do Reset	Assíncrono, ativo a *High*	Inicialização da máquina de estados
BtnStart	1	Tecla BTNC → Sincronizador	Assíncrono, ativo a *High*	Começa o jogo (carregar e largar)
Reset	1	Sincronizador do Reset → Controlador (PRE do FF_S0, CLR dos outros)	Ativação assíncrona, libertação síncrona	Põe a máquina em S0 de imediato
Start	1	Sincronizador → Controlador	Síncrono, ativo a *High*	Tecla Start sem ressaltos (10 ms)
Tempo	1	Temporizador variável → Controlador	Síncrono, ativo a *High*	1 enquanto o temporizador conta
ClrMem	1	Controlador → Memória e Beat (CLR)	Comando assíncrono (CLR)	Limpa a memória e apaga o LED Beat
InitLFSR	1	Controlador → LFSR (Init)	Comando síncrono	Carrega a semente

Sinal	Bits	Origem → destino	Natureza	Função
EnLFSR	1	Controlador → LFSR (Enable)	Comando síncrono	O LFSR avança um passo por ciclo
ShiftMem	1	Controlador → Memória (CE)	Comando síncrono	Desloca a memória e carrega a corda nova
RstTemp	1	Controlador → Temporizador variável	Comando síncrono	Para a contagem e repõe o período inicial
StartTemp	1	Controlador → Temporizador	Comando síncrono	Começa uma contagem com o período atual
Faster	1	Controlador → Registo da velocidade	Comando síncrono	Encurta o período (opcional)
TglBeat	1	Controlador → Beat (CE)	Comando síncrono	Troca o estado do LED Beat
Ready, Waiting	1 + 1	Controlador → LD0, LD2	Sinalização	Espera por Start; espera pelo fim do tempo
Estado[5:0]	6	Controlador → LD10 a LD15	Depuração	Saídas Q dos 6 *flip-flops*
OutLFSR[1:0]	2	LFSR → Memória	Dados, síncrono	Corda nova: 00 pausa, 01 cima, 10 meio, 11 baixo
Pos0 a Pos3	4 × 2	Memória → Gestão dos *displays*	Dados, síncrono	Corda de cada *display* (Pos0 à esquerda)
Periodo[27:0]	28	Registo da velocidade → Temporizador	Dados, síncrono	Tempo de um passo, em ciclos de 10 ns
Seg[6:0], DP	7 + 1	Gestão dos *displays* → cátodos	Ativos a *Low*	Segmentos do *display* ativo; ponto apagado
An[3:0]	4	Gestão dos *displays* → ânodos	Ativos a *Low*	Um *display* de cada vez (2 kHz)
LedBeat	1	Beat → LD1	Sinalização	Pisca ao ritmo do temporizador

3 Sincronização das teclas

3.1 Sincronizador e anti-ressalto da tecla Start

A tecla Start é assíncrona em relação ao relógio e tem ressaltos de alguns milissegundos. Ligada diretamente às portas das decisões de S1 e S2, poderia violar os tempos de preparação de FF_S1 e FF_S2 no mesmo flanco (um podia ver 0 e o outro 1, deixando a máquina com zero ou dois estados ativos), e um ressalto ao carregar faria passar de S2 a S3 antes de a tecla ser largada, começando o jogo cedo de mais.

O `Sincronizador` (Listagem 1) resolve os dois problemas: dois *flip-flops* em série (`meta` e `sinc`, com o atributo `ASYNC_REG` para o Vivado os colocar lado a lado) sincronizam a tecla, e a saída só muda depois de a entrada sincronizada se manter diferente dela durante `ESTAVEL` ciclos seguidos (10 ms). Qualquer ressalto reinicia a contagem.

Listagem 1: Sincronizador e anti-ressalto da tecla Start (`sincronizador/sincronizador.v`)

```
1  `timescale 1ns / 1ps
2
3  // Sincronizador e anti-ressalto da tecla Start.
4  // 1) Dois flip-flops em série sincronizam a tecla com o relógio de 100 MHz
5  //    (evitam a metaestabilidade, já que a tecla muda em qualquer instante;
6  //    ASYNC_REG pede ao Vivado para os colocar lado a lado).
7  // 2) A saída só muda depois de a entrada sincronizada se manter diferente dela durante
8  //    ESTAVEL ciclos seguidos; os ressaltos da tecla (mais curtos) são ignorados.
9  module Sincronizador #(
10     parameter ESTAVEL = 1000000     // 10 ms a 100 MHz
11     )(
12     input Clock,
13     input Entrada,
14     output reg Saida = 1'b0
15     );
16
17      (* ASYNC_REG = "TRUE" *) reg meta = 1'b0, sinc = 1'b0;  // cadeia de sincronização
18      reg [31:0] contagem = 0;       // ciclos seguidos com sinc diferente de Saida
19
20      always @(posedge Clock) begin
21          meta <= Entrada;
22          sinc <= meta;
23          if (sinc == Saida)
24              contagem <= 0;
25          else if (contagem == ESTAVEL - 1) begin
26              Saida <= sinc;
27              contagem <= 0;
28          end
29          else
30              contagem <= contagem + 1;
31      end
32
33 endmodule
```

O *testbench* (Listagem 14, no Anexo A) usa `ESTAVEL = 4` ciclos e aplica um impulso de ruído de 2 ciclos e depois carrega e larga a tecla com ressaltos. Na Figura 2, o ruído (10 a 30 ns) faz `contagem` chegar a 2 e voltar a 0, sem mudar a saída; ao carregar, cada ressalto põe `contagem` a 0 e `Saida` só sobe aos 245 ns, 6 flancos (`ESTAVEL` + 2 da sincronização) depois do último ressalto (190 ns); ao largar, desce da mesma forma aos 445 ns. Resultado: `Simulacao OK (7 verificacoes)`.

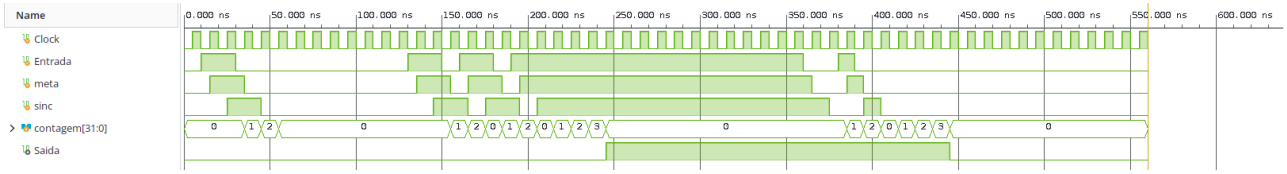


Figura 2: Simulação do sincronizador da tecla Start (ESTAVEL = 4 ciclos).

3.2 Sincronizador do Reset

O Reset tem de pôr a máquina em S0 de imediato, sem esperar pelo relógio (secção 2.7 do enunciado), mas a sua **libertação** não pode ser assíncrona: se a tecla fosse largada junto a um flanco, FF_S0 podia sair do PRE a tempo de carregar D = 0 nesse flanco enquanto FF_S1 ainda estava no CLR. A máquina ficava sem nenhum estado ativo e, como nenhuma equação de excitação tem termos fora dos estados, nunca mais sairia daí. É uma **corrida crítica** entre os sinais assíncronos (passo 5 da metodologia, secção 9.4).

O `SincronizadorReset` (Listagem 2) usa duas primitivas FDPE em série com `PRE` ligado à tecla: ao carregar, a saída vai a 1 de imediato (assíncrono); ao largar, só volta a 0 no 2.º flanco ascendente do relógio, pelo que os seis *flip-flops* saem do *reset* com um ciclo inteiro de folga. Com `INIT = 1`, o Reset também é libertado de forma síncrona depois de configurar a FPGA.

Listagem 2: Sincronizador do Reset (ativação assíncrona, libertação síncrona) (`sincronizador/sincronizadorreset.v`)

```
1  `timescale 1ns / 1ps
2
3  // Sincronizador da tecla Reset: ativação assíncrona, libertação síncrona.
4  // Dois flip-flops FDPE em série com PRE = Entrada:
5  //   - ao carregar Reset, Saida vai a 1 de imediato, sem esperar pelo relógio
6  //     (a máquina de estados é posta em S0 de forma assíncrona, secção 2.7);
7  //   - ao largar Reset, Saida só volta a 0 no 2.º flanco ascendente do relógio.
8  // Sem isto, se a tecla fosse largada mesmo junto a um flanco, FF_S0 podia sair do PRE
9  // (e carregar D = 0) antes de FF_S1 sair do CLR, e a máquina ficava sem nenhum estado ativo.
10 // INIT = 1: depois de configurar a FPGA o Reset também é libertado de forma síncrona.
11 module SincronizadorReset(
12     input Clock,
13     input Entrada,
14     output Saida
15     );
16
17      wire q1;
18
19      (* ASYNC_REG = "TRUE" *) FDPE #(.INIT(1'b1)) ff1
   (.Q(q1),    .C(Clock), .CE(1'b1), .PRE(Entrada), .D(1'b0));
20      (* ASYNC_REG = "TRUE" *) FDPE #(.INIT(1'b1)) ff2
   (.Q(Saida), .C(Clock), .CE(1'b1), .PRE(Entrada), .D(q1));
21
22 endmodule
```

O *testbench* (Listagem 15, no Anexo A) confere a saída depois de configurar a FPGA e ao carregar e largar a tecla entre dois flancos, incluindo um impulso de 2 ns. Na Figura 3, `q1` e `Saida` estão a 1 durante as configurações iniciais (100 ns) e descem nos flancos de 105 e 115 ns; a tecla carregada aos 132 ns põe `Saida` a 1 de imediato e, largada aos 153 ns, `Saida` só desce no 2.º flanco (165 ns); o impulso de 2 ns aos 182 ns também ativa o Reset, libertado no flanco de 195 ns. Resultado: `Simulacao OK (9 verificacoes)`.

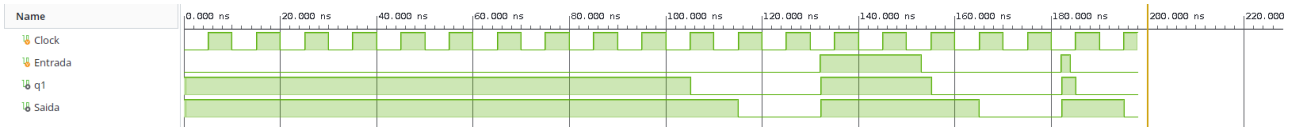


Figura 3: Simulação do sincronizador do Reset: subida imediata e descida síncrona.

4 LFSR de 32 bits

O `LFSR32` (Listagem 3) é o código do enunciado, com comentários. A realimentação é a XNOR das saídas 32, 22, 2 e 1 (*bits* 31, 21, 1 e 0), a combinação da nota de aplicação XAPP052 da Xilinx para 32 *bits*, o que dá o período máximo de $2^{32} - 1$ estados. Com XNOR o estado proibido é tudo a 1 (o LFSR ficaria parado); a semente `32'h03E3180A` é diferente desse valor.

O LFSR é comandado pela máquina de estados: com `Enable` a 1 carrega a semente se `Init` estiver a 1 e avança um passo se `Init` estiver a 0; com `Enable` a 0 mantém o estado. Como `Init` só atua com `Enable` a 1, o estado S0 ativa as duas saídas (InitLFSR e EnLFSR). A saída `OutLFSR = {regLFSR[31], regLFSR[0]}` escolhe a corda (Tabela 3).

Tabela 3: Correspondência entre os 2 bits do LFSR e as cordas.

OutLFSR	Corda	Segmento aceso	Seg[6:0] (a…g)
`00`	Pausa	Nenhum	`1111111`
`01`	Corda de cima	a	`0111111`
`10`	Corda do meio	g	`1111110`
`11`	Corda de baixo	d	`1110111`

O LFSR avança em todos os ciclos do relógio (EnLFSR está ativa em todos os estados), mas a memória só lê um par de *bits* em cada passo do jogo. O número de ciclos que a máquina passa em S1 e S2 depende do tempo que o jogador demora a carregar e largar Start, por isso a sequência de cordas é diferente em cada jogo, apesar de a semente ser fixa.

Listagem 3: LFSR de 32 bits (código do enunciado) (`lfsr32/lfsr32.v`)

```
`timescale 1ns / 1ps

// LFSR de 32 bits que gera a sequência pseudo-aleatória das cordas a tocar.
// Código do enunciado (secção 2.2), com realimentação XNOR nas saídas 32, 22, 2 e 1
// (tabela da nota de aplicação Xilinx XAPP052), o que dá um período de 2^32 - 1 ciclos.
// Com XNOR o estado proibido é 32'hFFFFFFFF (o LFSR ficaria parado), por isso a semente
// tem de ser diferente de tudo a 1; o estado 0 é válido.
// Init e Enable são comandados pela máquina de estados (InitLFSR e EnLFSR):
//   Enable = 1 e Init = 1: carrega a semente;
//   Enable = 1 e Init = 0: avança um passo em cada ciclo de relógio;
//   Enable = 0:            mantém o estado.
// OutLFSR (2 bits) escolhe a corda: 00 pausa, 01 cima, 10 meio, 11 baixo.
module LFSR32 #(
    parameter SEED = 32'h03E3180A   // semente aleatória, diferente de 32'hFFFFFFFF
    )(
    input Clock,
    input Enable,
    input Init,
    output [1:0] OutLFSR
    );

     reg [31:0] regLFSR = 0;
     reg regXNOR = 1;

//   Registo de deslocamento: carrega a semente ou desloca para a esquerda
     always @(posedge Clock)
         if (Enable) begin
             if (Init)
                 regLFSR <= SEED;
             else
                 regLFSR[31:0] <= {regLFSR[30:0], regXNOR};
         end

```

```
34  //   Realimentação: XNOR das saídas 32, 22, 2 e 1 (bits 31, 21, 1 e 0)
35       always @(*)
36           regXNOR = regLFSR[31] ~^ regLFSR[21] ~^ regLFSR[1] ~^ regLFSR[0];
37
38       assign OutLFSR = {regLFSR[31], regLFSR[0]};
39
40  endmodule
```

O *testbench* (Listagem 16, no Anexo A) confere o registo e a saída com valores calculados à mão: estado inicial 0, `Init` sem efeito com `Enable` a 0, carga da semente, três passos, `Enable` a 0 a manter o estado e o regresso ao avanço. Na Figura 4 a semente é carregada aos 25 ns e os passos seguintes dão `07c63015`, `0f8c602a` e `1f18c054` (entra à direita o valor de `regXNOR`); com `Enable` a 0 o registo não muda nos flancos de 65, 75 e 85 ns e depois volta a avançar. Resultado: `Simulacao OK (9 verificacoes)`.

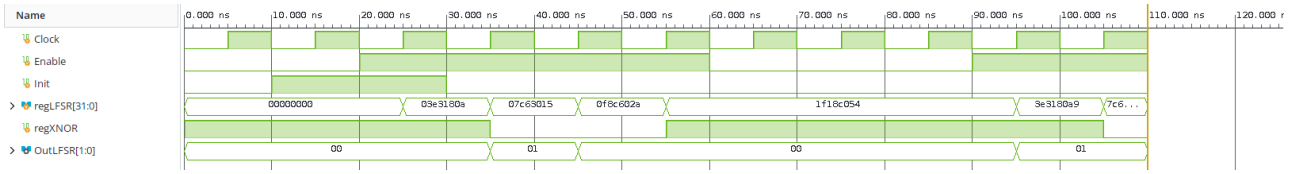


Figura 4: Simulação do LFSR: carga da semente, três passos, Enable = 0 e novo avanço.

5 Memória 4x2

A memória guarda a corda de cada *display* e funciona como uma fila (registo de deslocamento): com `Shift` a 1, no flanco ascendente do relógio, `Pos0` recebe os 2 *bits* do LFSR e cada posição recebe a anterior, perdendo-se a de `Pos3`. `CLR` limpa as quatro posições de forma assíncrona (todas em pausa).

A `Memoria4x2` (Listagem 4) reutiliza quatro `Registo4` do 1.º trabalho (quatro primitivas FDCE cada), em cadeia, dos quais só se usam os 2 *bits* de menor peso: os de maior peso carregam sempre 0 e a síntese elimina-os.

Listagem 4: Memória 4x2 com quatro registos do 1.º trabalho (`memoria4x2/memoria4x2.v`)

```
`timescale 1ns / 1ps

// Memória 4x2 com as cordas dos 4 displays, organizada como uma fila (registo de deslocamento).
// Feita com 4 Registo4 do 1.º trabalho, dos quais só se usam os 2 bits de menor peso
// (os 2 de maior peso carregam sempre 0 e a síntese elimina-os).
// Com Shift a 1, no flanco ascendente do relógio:
//   Pos0 <- Data (2 bits do LFSR), Pos1 <- Pos0, Pos2 <- Pos1, Pos3 <- Pos2
//   (perde-se o valor que estava em Pos3).
// CLR limpa as 4 posições de forma assíncrona (todas as cordas em pausa).
module Memoria4x2(
    input [1:0] Data,
    input Shift,
    input Clk,
    input CLR,
    output [1:0] Pos0,
    output [1:0] Pos1,
    output [1:0] Pos2,
    output [1:0] Pos3
    );

//   Conteúdo dos 4 registos (4 bits cada)
     wire [3:0] Q0, Q1, Q2, Q3;

//   Fila: cada registo carrega o anterior e o primeiro carrega a corda nova
     Registo4 reg0 (.Data({2'b00, Data}), .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q0));
     Registo4 reg1 (.Data(Q0),            .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q1));
     Registo4 reg2 (.Data(Q1),            .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q2));
     Registo4 reg3 (.Data(Q2),            .CE(Shift), .Clk(Clk), .CLR(CLR), .DataOut(Q3));

     assign Pos0 = Q0[1:0];
     assign Pos1 = Q1[1:0];
     assign Pos2 = Q2[1:0];
     assign Pos3 = Q3[1:0];

endmodule
```

O *testbench* (Listagem 17, no Anexo A) confere as quatro posições depois de cada flanco: cinco cordas a entrar, `Shift` a 0 a manter o conteúdo e `CLR` fora do flanco e com prioridade sobre `Shift`. Na Figura 5 as cordas 01, 10, 11, 00 e 10 entram em `Pos0` e avançam uma posição em cada flanco (no 5.º flanco o 01 sai de `Pos3`); aos 190 ns o `CLR` limpa tudo sem esperar pelo relógio e, depois de voltar a 0 (210 ns), a corda 01 entra no flanco seguinte. Resultado: `Simulacao OK (10 verificacoes)`.

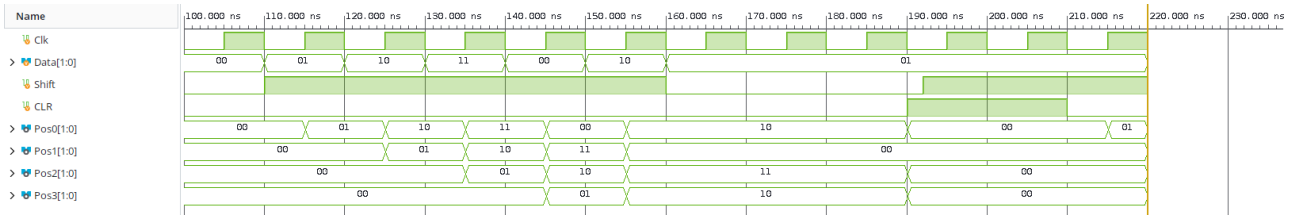


Figura 5: Simulação da memória 4x2: deslocamento das cordas e clear assíncrono.

6 Temporizador variável

O temporizador variável define a duração de cada passo do jogo. Foi dividido em dois módulos, ligados pelo `TemporizadorVariavel` (Listagem 7):

- o `Temporizador` (Listagem 5), pedido na secção 2.3 do enunciado: com `Start` a 1 carrega `Periodo` (o tempo total, em ciclos de 10 ns) e `Tempo` fica a 1 durante `Periodo` ciclos, voltando a 0 sozinho; `Reset` interrompe a contagem;
- o registo da velocidade, `Velocidade` (Listagem 6), que guarda `Periodo`: `Reset` repõe o valor inicial (0,5 s) e cada impulso de `Faster` encurta-o em 5 ms, sem descer abaixo de 0,2 s (parte opcional do enunciado).

Com estes valores o jogo chega à velocidade máxima ao fim de 60 passos (cerca de 21 s). Cada passo dura `Periodo` + 3 ciclos do relógio (estados S3 e S5 e o ciclo de S4 em que Tempo já está a 0), ou seja, mais 30 ns do que o período, o que é desprezável. O contador tem 28 *bits* (até 2,68 s).

Listagem 5: Temporizador (`temporizador/temporizador.v`)

```
`timescale 1ns / 1ps

// Temporizador (secção 2.3 do enunciado).
// Periodo é o tempo total a temporizar, em ciclos do relógio de 100 MHz (10 ns).
// Com Start a 1, no flanco ascendente do relógio, carrega Periodo e começa a contar;
// Tempo fica a 1 durante Periodo ciclos e volta a 0 sozinho no fim da contagem.
// Reset (síncrono) interrompe a contagem e inicializa o temporizador (Tempo = 0).
module Temporizador #(
    parameter N = 28               // bits do contador: até 2^28 ciclos = 2,68 s
    )(
    input Clock,
    input Reset,
    input Start,
    input [N-1:0] Periodo,
    output Tempo
    );

     reg [N-1:0] contagem = 0;      // ciclos que faltam

     always @(posedge Clock)
         if (Reset)
             contagem <= 0;
         else if (Start)
             contagem <= Periodo;
         else if (contagem != 0)
             contagem <= contagem - 1;

     assign Tempo = (contagem != 0);

endmodule
```

Listagem 6: Registo da velocidade do jogo (`temporizador/velocidade.v`)

```
`timescale 1ns / 1ps

// Registo da velocidade do jogo: guarda o período do temporizador (em ciclos de 10 ns).
// Reset (síncrono) repõe o período inicial.
// Com Faster a 1 o período diminui PASSO ciclos no flanco ascendente do relógio,
// sem descer abaixo de MINIMO (parte opcional do enunciado: acelerar ligeiramente).
module Velocidade #(
    parameter N       = 28,
    parameter INICIAL = 50000000,   // 0,5 s por passo da sequência
    parameter PASSO   = 500000,     // acelera 5 ms em cada passo
```

```
    parameter MINIMO  = 20000000    // velocidade máxima: 0,2 s por passo
    )(
    input Clock,
    input Reset,
    input Faster,
    output reg [N-1:0] Periodo = INICIAL
    );

     always @(posedge Clock)
         if (Reset)
             Periodo <= INICIAL;
         else if (Faster && Periodo >= MINIMO + PASSO)
             Periodo <= Periodo - PASSO;

endmodule
```

Listagem 7: Temporizador variável (`temporizador/temporizadorvariavel.v`)

```
`timescale 1ns / 1ps

// Temporizador variável: o registo da velocidade fornece ao temporizador o tempo total a
// temporizar (Periodo). É o bloco "Temporizador variável" da arquitetura a controlar.
//   RstTemp:   inicializa os dois (período inicial e contagem parada);
//   StartTemp: começa uma contagem com o período atual;
//   Faster:    encurta o período para as próximas contagens;
//   Tempo:     1 enquanto a contagem decorre.
module TemporizadorVariavel #(
    parameter N       = 28,
    parameter INICIAL = 50000000,
    parameter PASSO   = 500000,
    parameter MINIMO  = 20000000
    )(
    input Clock,
    input RstTemp,
    input StartTemp,
    input Faster,
    output Tempo
    );

     wire [N-1:0] Periodo;

     Velocidade #(.N(N), .INICIAL(INICIAL), .PASSO(PASSO), .MINIMO(MINIMO)) velocidade (
         .Clock(Clock),
         .Reset(RstTemp),
         .Faster(Faster),
         .Periodo(Periodo)
     );

     Temporizador #(.N(N)) temporizador (
         .Clock(Clock),
         .Reset(RstTemp),
         .Start(StartTemp),
         .Periodo(Periodo),
         .Tempo(Tempo)
     );

endmodule
```

O *testbench* do temporizador (Listagem 18, no Anexo A) usa um contador de 8 *bits* e confere `Tempo` ciclo a ciclo: com `Periodo` = 3 fica a 1 durante 3 ciclos e volta a 0 sozinho; com `Periodo` = 0 não sobe; `Reset`

interrompe a contagem e tem prioridade sobre `Start`; um segundo `Start` a meio da contagem recomeça com o novo período. Na Figura 6 `contagem` desce de 3 até 0 (15 a 45 ns), o `Reset` aos 120 ns para uma contagem de 5 e, aos 195 ns, o `Start` com `Periodo` = 4 recomeça uma contagem de 3 que ainda ia em 2, pelo que `Tempo` só volta a 0 aos 235 ns. Resultado: `Simulacao OK (18 verificacoes)`.

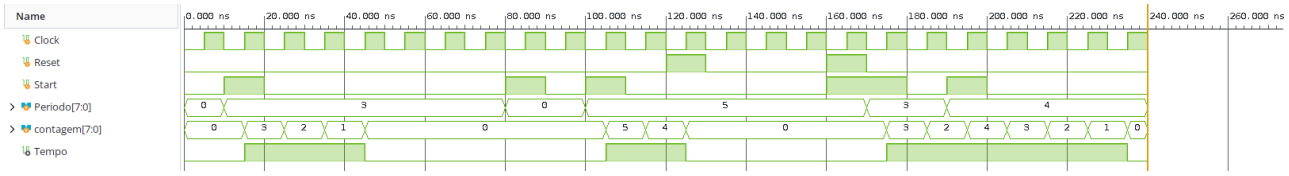


Figura 6: Simulação do temporizador: período de 3 ciclos, Reset e novo Start a meio da contagem.

O *testbench* do temporizador variável (Listagem 19, no Anexo A) usa `INICIAL = 5`, `PASSO = 1` e `MINIMO = 3` ciclos e confere o período guardado no registo da velocidade e a saída `Tempo`. Na Figura 7, depois de `RstTemp`, `StartTemp` põe `Tempo` a 1 durante 5 ciclos; cada impulso de `Faster` encurta o período (5, 4, 3), que fica em 3 no segundo ciclo de `Faster` (3 − 1 seria menor do que o mínimo); a contagem seguinte dura 3 ciclos e `RstTemp` a meio de outra contagem para o temporizador e repõe os 5 ciclos. Resultado: `Simulacao OK (14 verificacoes)`.

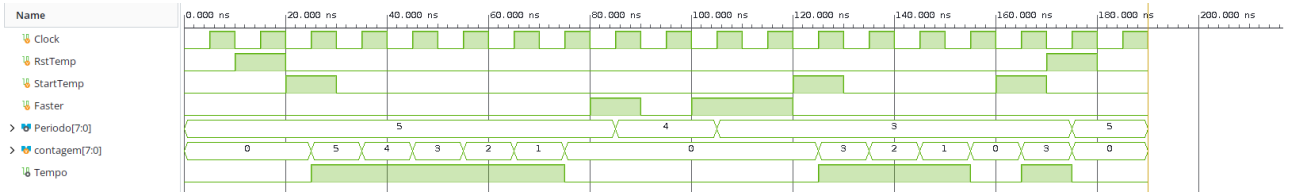


Figura 7: Simulação do temporizador variável: aceleração até ao período mínimo e reposição com RstTemp.

7 LED Beat

O LED Beat pisca ao ritmo do temporizador: o `Beat` (Listagem 8) é um *flip-flop* T feito com uma primitiva FDCE com `D` = /`Q` e `CE` = `Tgl`. Em cada passo do jogo o controlador ativa TglBeat durante um ciclo e o LED troca de estado (aceso num passo, apagado no seguinte). `CLR` está ligado a ClrMem, pelo que o LED apaga com a inicialização. Assim basta um único comando, sem *latch* SR.

Listagem 8: Flip-flop T do LED Beat (`beat/beat.v`)

```
1  `timescale 1ns / 1ps
2
3  // Flip-flop T do LED Beat, feito com uma primitiva FDCE com D = /Q.
4  // Em cada flanco ascendente do relógio com Tgl a 1 o LED troca de estado,
5  // por isso pisca ao ritmo do temporizador variável (Tgl = TglBeat, uma vez por passo).
6  // CLR apaga o LED de forma assíncrona.
7  module Beat(
8      input Clk,
9      input Tgl,
10     input CLR,
11     output Q
12     );
13
14      wire nQ;
15
16      not inv (nQ, Q);
17      FDCE #(.INIT(1'b0)) ff (.Q(Q), .C(Clk), .CE(Tgl), .CLR(CLR), .D(nQ));
18
19 endmodule
```

O *testbench* (Listagem 20, no Anexo A) confere que `Q` troca em cada flanco com `Tgl` a 1, mantém o valor com `Tgl` a 0 e vai a 0 com `CLR` fora do flanco (170 ns), mesmo com `Tgl` a 1, voltando a trocar quando `CLR` volta a 0 (Figura 8). Resultado: `Simulacao OK (8 verificacoes)`.

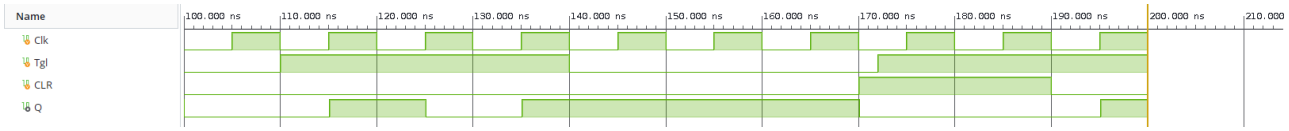


Figura 8: Simulação do flip-flop T do LED Beat.

8 Gestão dos displays

A gestão dos *displays* (Listagem 10) reutiliza o sistema do 1.º trabalho (secção 2.5 do enunciado): o `DivisorClock` e o `Contador4` fazem o varrimento a 2 kHz (cada *display* aceso 0,5 ms, os quatro atualizados a cada 2 ms), o `multiplexer16x4` escolhe a corda do *display* ativo e o `descodificador2x4`, com os *enables* a 1, ativa um *display* de cada vez. Os registos dos símbolos foram substituídos pela memória 4x2, cujas saídas entram diretamente no *multiplexer*.

O transcodificador foi alterado para as cordas (`TranscodCordas`, Listagem 9 e Tabela 3): cada corda acende um só segmento horizontal (a, g ou d) e a pausa apaga o *display*. A corda nova (`Pos0`) aparece no *display* da esquerda (AN3) e avança para a direita até `Pos3` (AN0). Segmentos e ânodos são ativos a *Low* e o ponto decimal fica apagado.

Listagem 9: Transcodificador das cordas (`gestaodisplays/transcodcordas.v`)

```
`timescale 1ns / 1ps

// Transcodificador das cordas para os segmentos horizontais do display.
// Versão alterada do Transcod7Seg do 1.º trabalho (secção 2.5 do enunciado):
// cada corda acende um só segmento horizontal e a pausa apaga o display.
// Segmentos[6] = a ... Segmentos[0] = g, ativos a Low (0 = segmento aceso).
module TranscodCordas(
    input [1:0] Corda,
    output reg [6:0] Segmentos
    );

     always @(*)
         case (Corda)                //  abcdefg
             2'b00:   Segmentos = 7'b1111111;   // pausa: tudo apagado
             2'b01:   Segmentos = 7'b0111111;   // corda de cima (a)
             2'b10:   Segmentos = 7'b1111110;   // corda do meio (g)
             2'b11:   Segmentos = 7'b1110111;   // corda de baixo (d)
             default: Segmentos = 7'b1111111;   // nunca usado: os 4 casos estão cobertos
         endcase

endmodule
```

Listagem 10: Gestão dos quatro displays (`gestaodisplays/gestaodisplays.v`)

```
`timescale 1ns / 1ps

// Gestão dos 4 displays do jogo, feita com o sistema do 1.º trabalho (secção 2.5):
//   - DivisorClock + Contador4: varrimento dos displays (2 kHz);
//   - multiplexer16x4: corda do display ativo para o transcodificador;
//   - TranscodCordas: transcodificador alterado (um segmento horizontal por corda);
//   - descodificador2x4: ativa um display de cada vez (enables sempre a 1).
// Os registos dos símbolos do 1.º trabalho passam a ser a memória da sequência (Memoria4x2),
// por isso as cordas entram diretamente em Pos0 a Pos3.
// A corda nova (Pos0) aparece no display da esquerda (Anodos[3]) e avança para a direita
// até Pos3, no display da direita (Anodos[0]).
// Segmentos, ponto decimal e ânodos são ativos a Low na Basys3.
module GestaoDisplays #(
    parameter METADE = 25000        // varrimento a 100 MHz / (2 x 25000) = 2 kHz
    )(
    input [1:0] Pos0,
    input [1:0] Pos1,
    input [1:0] Pos2,
    input [1:0] Pos3,
    input Clock,
    output [6:0] Segmentos,
    output DotPoint,
```

```
    output [3:0] Anodos
    );

//   Sinais internos
     wire [3:0] ValorAtivo;                     // corda do display ativo (2 bits de menor peso)
     wire [3:0] DisplayAtivo;                   // saídas do descodificador (ativas a High)
     wire ClockVarrimento;                      // relógio dividido
     wire Control1, Control0;                   // variáveis de varrimento

//   Varrimento: divisor de clock e contador de módulo 4
     DivisorClock #(.METADE(METADE)) divisor (
         .ClockIn(Clock),
         .ClockOut(ClockVarrimento)
     );

     Contador4 contador (
         .Clock(ClockVarrimento),
         .Q1(Control1),
         .Q0(Control0)
     );

//   Multiplexer 16x4: o display i mostra a entrada i (display 3 = esquerda = Pos0)
     multiplexer16x4 mux16x4 (
         .DataIn0({2'b00, Pos3}),
         .DataIn1({2'b00, Pos2}),
         .DataIn2({2'b00, Pos1}),
         .DataIn3({2'b00, Pos0}),
         .DataOut(ValorAtivo),
         .Sel({Control1, Control0})
     );

//   Transcodificador das cordas: segmentos partilhados por todos os displays (ativos a Low)
     TranscodCordas transcod (
         .Corda(ValorAtivo[1:0]),
         .Segmentos(Segmentos)
     );

     assign DotPoint = 1'b1;            // ponto decimal sempre apagado

//   Descodificador 2x4: ativa um display de cada vez
     descodificador2x4 descod (
         .Bit1(Control1),
         .Bit0(Control0),
         .En0(1'b1),
         .En1(1'b1),
         .En2(1'b1),
         .En3(1'b1),
         .Out0(DisplayAtivo[0]),
         .Out1(DisplayAtivo[1]),
         .Out2(DisplayAtivo[2]),
         .Out3(DisplayAtivo[3])
     );

     assign Anodos = ~DisplayAtivo;     // ânodos comuns ligados por transistores PNP: ativos a Low

endmodule
```

O *testbench* (Listagem 21, no Anexo A) usa `METADE = 2` (o *display* ativo muda a cada 40 ns) e põe nas quatro posições as cordas de cima, pausa, de baixo e do meio; em cada passo do varrimento confere o ânodo ativo, os segmentos e o ponto decimal. Depois as cordas avançam uma posição (entra a corda de

baixo em `Pos0`) e confere mais uma volta. Na Figura 9, com `Pos0` a `Pos3` = 01, 00, 11 e 10, `Anodos` percorre 1110, 1101, 1011 e 0111 e `Segmentos` mostra, respetivamente, a corda do meio (1111110), a de baixo (1110111), a pausa (1111111) e a de cima (0111111): `Pos3` aparece no *display* da direita e `Pos0` no da esquerda. Resultado: `Simulacao OK (9 verificacoes)`.

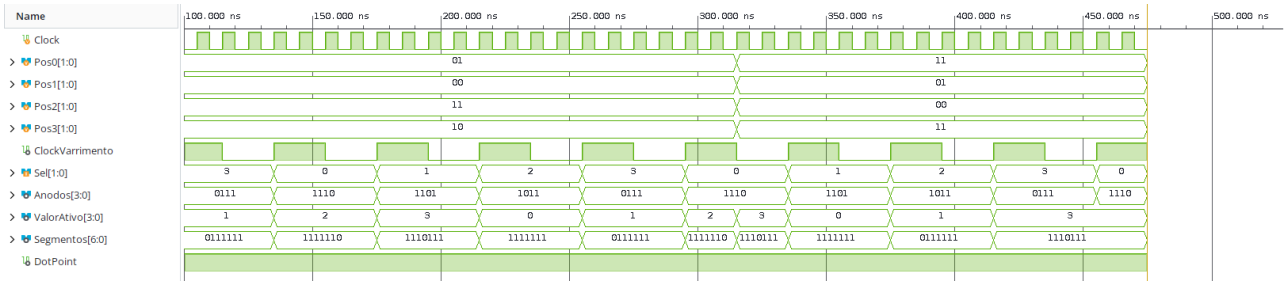


Figura 9: Simulação da gestão dos displays (METADE = 2): varrimento antes e depois de as cordas avançarem uma posição.

9 Controlador: máquina de estados por realização direta

O controlador foi projetado pela metodologia lecionada, nos 11 passos da Tabela 4. A metodologia foi apresentada para máquinas assíncronas; em cada passo indica-se o que muda numa máquina **síncrona** por realização direta, em que todos os *flip-flops* mudam no mesmo flanco do relógio de 100 MHz.

Tabela 4: Passos do projeto do controlador.

Passo	O que se faz	Secção
1	Construir a tabela de estados primitiva	9.2
2	Eliminar estados redundantes, por inspeção visual e por partições	9.3
3	Agregar estados compatíveis pelo método de fusão	9.3
4	Desenhar a tabela de estados reduzida	9.3
5	Determinar uma codificação dos estados livre de corridas críticas	9.4
6	Desenhar a tabela de transições (excitação)	9.5
7	Determinar o valor das saídas nas transições, sem picos	9.6
8 e 9	Equações de excitação e de saída, sem acidentes	9.7
10	Desenhar o diagrama lógico, com as realimentações	9.8
11	Verificar e testar o circuito	9.9 e 10

9.1 Algoritmo e fluxograma

O fluxograma da Figura 10 foi desenhado a partir dos sinais identificados na secção 2.3: as decisões testam as entradas do controlador (Start e Tempo) e cada estado ativa as saídas indicadas no seu interior (saídas de Moore). O código ao lado de cada estado é o valor de Q5…Q0 (um *flip-flop* por estado).

- **S0** (só com Reset): limpa a memória e o LED Beat, carrega a semente no LFSR e inicializa o temporizador variável.
- **S1**: LED Ready aceso; o LFSR avança livremente; espera que se carregue Start.
- **S2**: Ready continua aceso; espera que se largue Start.
- **S3** (um ciclo): Ready apaga-se; desloca a memória carregando 2 *bits* do LFSR em `Pos0`; troca o LED Beat; começa a contagem do temporizador.
- **S4**: LED Waiting aceso enquanto o temporizador conta (Tempo = 1).
- **S5** (opcional, um ciclo): acelera ligeiramente o temporizador e repete a partir de S3. Sem a parte opcional, o ramo Tempo = 0 iria diretamente para S3.

Cada bloco tem um só estado, no máximo uma decisão e um único estado seguinte para cada combinação das entradas, pelo que o fluxograma é válido. O Reset não aparece como decisão porque atua de forma assíncrona, nos PRE e CLR dos *flip-flops*. Se a tecla Start estiver carregada no momento em que se larga Reset, S1 passa logo a S2 e o jogo começa quando Start for largada: o fluxograma só exige ver a tecla carregada e depois largada.

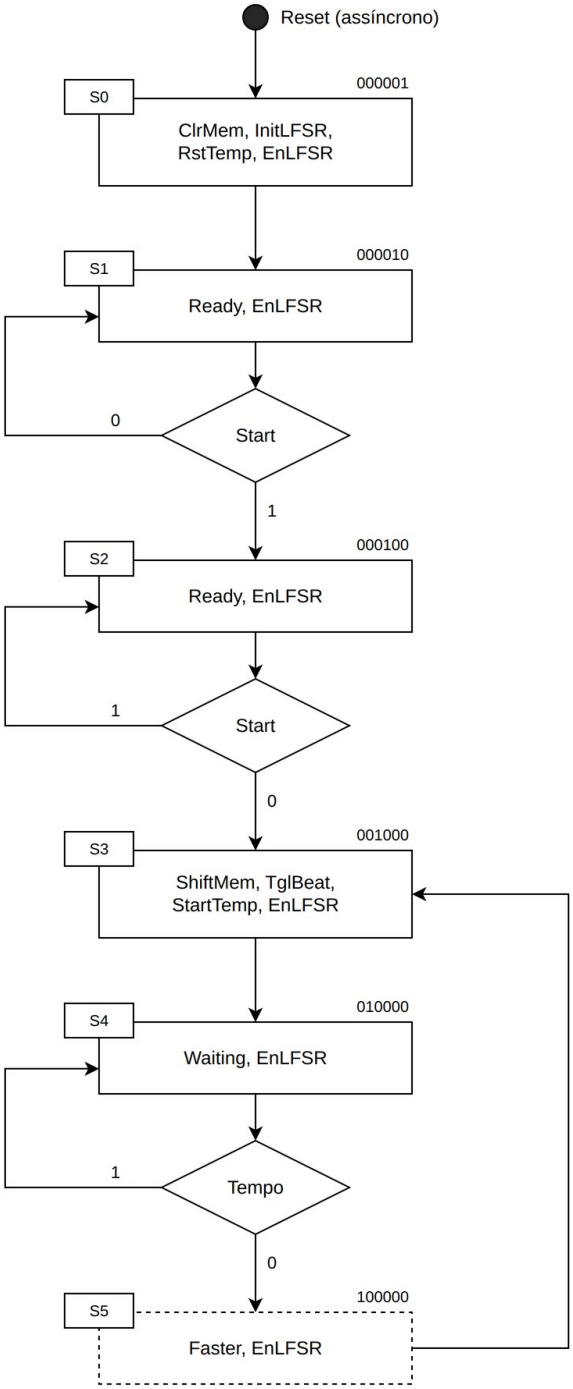


Figura 10: Fluxograma do algoritmo de controlo (S5, a tracejado, é a parte opcional).

9.2 Tabela de estados primitiva (passo 1)

A tabela de estados primitiva (Tabela 5) foi lida diretamente do fluxograma: uma linha por estado e uma coluna por combinação das entradas testadas (Start Tempo, em código de Gray). As células a sombreado indicam que o estado se mantém (estado estável para essa combinação). Start só interessa em S1 e S2 e Tempo só em S4: nas outras linhas o estado seguinte é igual nas quatro colunas. A Figura 11 mostra o diagrama de estados equivalente.

Tabela 5: Tabela de estados primitiva (estado seguinte para Start Tempo = 00, 01, 11 e 10; saídas de Moore).

Estado presente	00	01	11	10	ClrMem	InitLFSR	EnLFSR	ShiftMem	RstTemp	StartTemp	Faster	TglBeat	Ready	Waiting
S0	S1	S1	S1	S1	1	1	1	0	1	0	0	0	0	0
S1	**S1**	**S1**	S2	S2	0	0	1	0	0	0	0	0	1	0
S2	S3	S3	**S2**	**S2**	0	0	1	0	0	0	0	0	1	0
S3	S4	S4	S4	S4	0	0	1	1	0	1	0	1	0	0
S4	S5	**S4**	**S4**	S5	0	0	1	0	0	0	0	0	0	1
S5	S3	S3	S3	S3	0	0	1	0	0	0	1	0	0	0

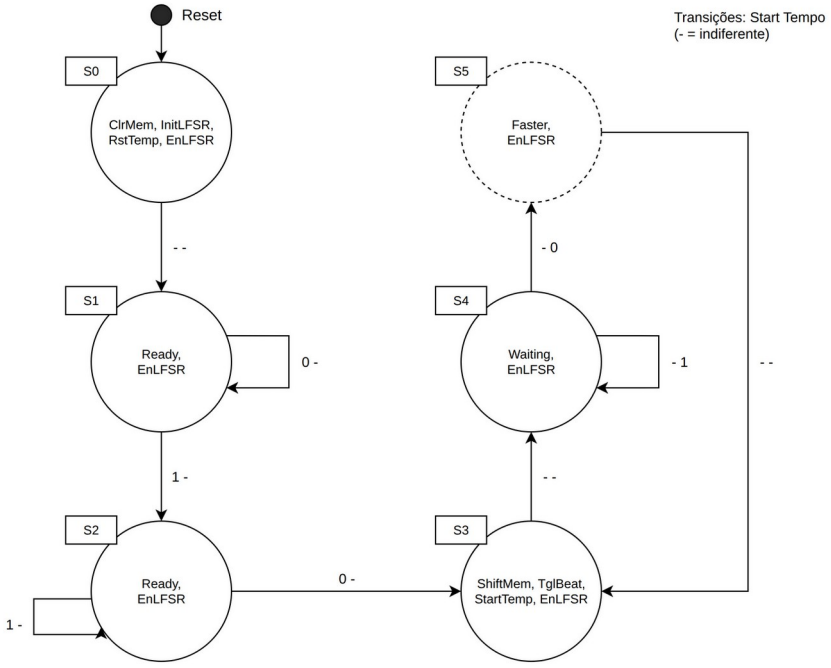


Figura 11: Diagrama de estados (saídas a 1 no interior de cada estado; transições etiquetadas com Start Tempo).

9.3 Redução de estados (passos 2 a 4)

Inspeção visual. Dois estados seriam redundantes se tivessem as mesmas saídas e o mesmo estado seguinte em todas as colunas. Não há nenhum par nessas condições: S1 e S2 têm as mesmas saídas, mas estados seguintes diferentes.

Partições. Começa-se com todos os estados num só bloco, separam-se os que têm saídas diferentes e, depois, os que vão para blocos diferentes com a mesma combinação das entradas, até a partição não mudar (Tabela 6).

Tabela 6: Redução de estados pelo método das partições.

Partição	Justificação
`P0 = (S0 S1 S2 S3 S4 S5)`	Todos os estados num só bloco
`P1 = (S0) (S1 S2) (S3) (S4) (S5)`	Separados pelas saídas: só S1 e S2 têm as mesmas
`P2 = (S0) (S1) (S2) (S3) (S4) (S5)`	Com Start Tempo = 00, S1 vai para S1, no bloco (S1 S2), e S2 vai para S3, no bloco (S3)
`P3 = P2`	A partição não muda: 6 classes, 6 estados

Fusão (passo 3). A tabela está completamente especificada (não há estados seguintes nem saídas indiferentes), pelo que dois estados só são compatíveis se forem equivalentes. O único par com as

mesmas saídas, (S1, S2), já foi separado no passo 2: o grafo de fusão não tem arestas e não há estados a agregar.

Tabela reduzida (passo 4). É a própria tabela primitiva: os seis estados são todos necessários. S1 e S2 esperam, respetivamente, que se carregue e que se largue Start; S3 (deslocar) e S5 (acelerar) duram um só ciclo do relógio; S4 espera pelo fim do tempo; S0 inicializa a arquitetura.

9.4 Codificação dos estados (passo 5)

O enunciado impõe a **realização direta, com um *flip-flop* por estado** (codificação *one-hot*): 6 *flip-flops* em vez dos 3 de uma codificação mínima, mas com uma correspondência unívoca entre cada *flip-flop* e um estado do fluxograma (Tabela 7). FF_S0 é uma primitiva FDPE (com *preset*), porque o Reset tem de o pôr a 1; os restantes são FDCE (com *clear*).

Tabela 7: Codificação dos estados e primitivas usadas.

Estado	Q5 Q4 Q3 Q2 Q1 Q0	Flip-flop	Primitiva	Reset	Valor inicial
S0	`0 0 0 0 0 1`	FF_S0	FDPE	PRE	INIT = 1
S1	`0 0 0 0 1 0`	FF_S1	FDCE	CLR	INIT = 0
S2	`0 0 0 1 0 0`	FF_S2	FDCE	CLR	INIT = 0
S3	`0 0 1 0 0 0`	FF_S3	FDCE	CLR	INIT = 0
S4	`0 1 0 0 0 0`	FF_S4	FDCE	CLR	INIT = 0
S5	`1 0 0 0 0 0`	FF_S5	FDCE	CLR	INIT = 0

Corridas. Em cada transição mudam dois *flip-flops* (o do estado que sai vai a 0 e o do estado que entra vai a 1). Numa máquina assíncrona isto seria uma corrida; aqui todos os *flip-flops* mudam no mesmo flanco do relógio e as entradas D só voltam a ser lidas no flanco seguinte, já estáveis (o Vivado indica uma folga de 5,085 ns em 10 ns). Por isso **não há corridas críticas** na evolução normal da máquina.

O único acontecimento assíncrono é a libertação do Reset, que atua nos PRE e CLR. Se FF_S0 saísse do PRE antes de FF_S1 sair do CLR, a máquina ficava em 000000 sem sair de lá. Esta corrida crítica foi eliminada com o sincronizador do Reset (secção 3.2), que só liberta o Reset num flanco do relógio, com um ciclo inteiro de folga para todos os *flip-flops*.

9.5 Tabela de transições (passo 6)

Com *flip-flops* D a excitação é o próprio estado seguinte (D = Q^+). A Tabela 8 tem uma linha por transição do fluxograma e indica o termo do estado seguinte que lhe corresponde. As outras 58 combinações de Q5…Q0 nunca ocorrem, porque há sempre um só *flip-flop* a 1.

Tabela 8: Tabela de transições (– = indiferente).

Estado presente	Q5…Q0	Start Tempo	Estado seguinte	D5…D0	Termo
S0	`000001`	`– –`	S1	`000010`	S0
S1	`000010`	`0 –`	S1	`000010`	S1·/Start
S1	`000010`	`1 –`	S2	`000100`	S1·Start
S2	`000100`	`1 –`	S2	`000100`	S2·Start
S2	`000100`	`0 –`	S3	`001000`	S2·/Start
S3	`001000`	`– –`	S4	`010000`	S3
S4	`010000`	`– 1`	S4	`010000`	S4·Tempo
S4	`010000`	`– 0`	S5	`100000`	S4·/Tempo
S5	`100000`	`– –`	S3	`001000`	S5

9.6 Saídas nas transições (passo 7)

Sendo de Moore, as saídas só mudam logo a seguir ao flanco do relógio. A Tabela 9 analisa se alguma pode ter um pico numa transição e se isso teria efeito.

Tabela 9: Saídas do controlador nas transições.

Saída	Atua em	Pico numa transição?
ClrMem = S0	CLR **assíncrono** da memória e do Beat	Não: vem diretamente da saída Q de FF_S0
InitLFSR, RstTemp = S0	Entradas síncronas	Não (saída Q de FF_S0)
EnLFSR = S0 + … + S5	Enable síncrono do LFSR	Possível 0 muito curto entre dois flancos, sem efeito: o LFSR só o lê no flanco
Ready = S1 + S2	LED LD0	Possível 0 de nanossegundos de S1 para S2: invisível
ShiftMem, TglBeat, StartTemp = S3	CE e Start síncronos	Não (saída Q de FF_S3)
Waiting = S4, Faster = S5	LED e registo síncrono	Não (saída Q de um só *flip-flop*)

A única saída que atua de forma assíncrona (ClrMem, nos CLR dos registos) vem diretamente de um *flip-flop*, por isso nunca tem picos; as que passam por uma porta OR só vão para entradas síncronas ou para LEDs. Não foi preciso alterar o valor das saídas nas transições.

9.7 Equações de excitação e de saída (passos 8 e 9)

Cada D é a soma dos termos das linhas da tabela de transições que levam a esse estado. Em *one-hot* um único Q a 1 identifica o estado, pelo que cada termo é apenas o estado vezes a condição testada, sem mapas de Karnaugh a minimizar (Tabela 10). Num fluxograma isto lê-se diretamente: as decisões são portas AND (estado · condição) e a reunião dos percursos que entram num estado é uma porta OR.

Tabela 10: Equações de excitação.

Equação	Leitura no fluxograma
`D_S0 = 0`	Nenhum percurso entra em S0 (só a tecla Reset, pelo PRE)
`D_S1 = S0 + S1·/Start`	Vem de S0, ou fica em S1 enquanto Start = 0
`D_S2 = S1·Start + S2·Start`	Vem de S1, ou fica em S2 enquanto Start = 1
`D_S3 = S2·/Start + S5`	Vem de S2 (Start largada) ou de S5
`D_S4 = S3 + S4·Tempo`	Vem de S3, ou fica em S4 enquanto Tempo = 1
`D_S5 = S4·/Tempo`	Vem de S4 quando o tempo acaba (sem S5: D_S3 = S2·/Start + S4·/Tempo)

Sem acidentes. Um pico na lógica de um D não tem efeito, porque o *flip-flop* só lê D no flanco, depois de a lógica estabilizar; além disso Start chega já sincronizada e Tempo é síncrono, pelo que nenhuma entrada assíncrona chega a estas portas. As equações de saída lêem-se nas colunas das saídas da tabela primitiva (soma dos estados em que a saída está a 1):

Tabela 11: Equações de saída.

Saídas	Equação
ClrMem, InitLFSR, RstTemp	`S0`
EnLFSR	`S0 + S1 + S2 + S3 + S4 + S5`
Ready	`S1 + S2`
ShiftMem, TglBeat, StartTemp	`S3`
Waiting	`S4`
Faster	`S5`

9.8 Diagrama lógico e código Verilog estrutural (passo 10)

A Figura 12 é o diagrama lógico do controlador: um *flip-flop* por estado, com a lógica do estado seguinte à esquerda e as saídas de Moore à direita. As **realimentações** (saídas Q dos *flip-flops*, S0 a S5) voltam às portas das decisões pelo barramento desenhado a traço grosso. Os nomes debaixo das portas são os nomes das instâncias no código Verilog.

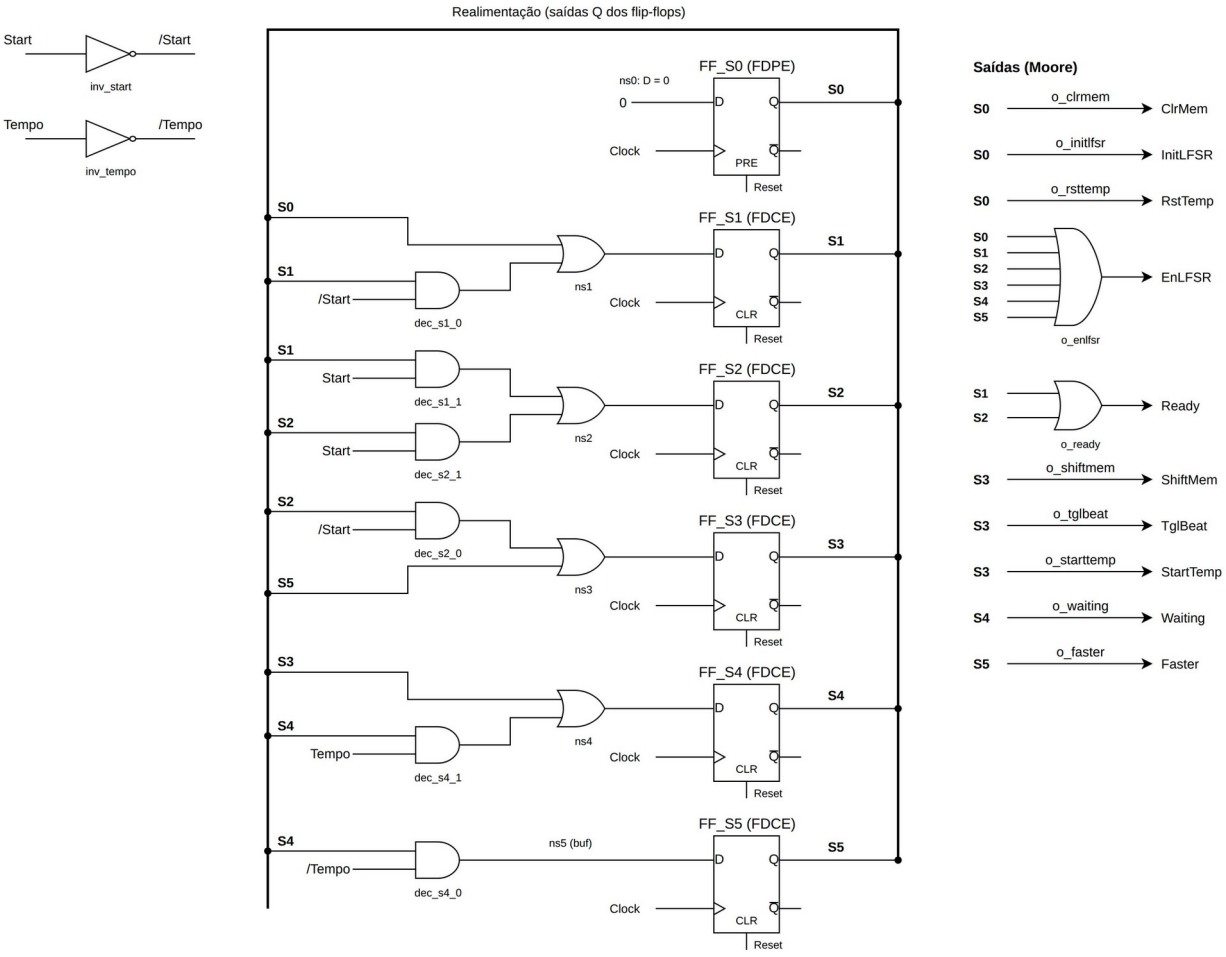


Figura 12: Diagrama lógico do controlador (realização direta), com as realimentações.

O código (Listagem 11) segue o **modelo estrutural** exigido no enunciado: cada estado é uma primitiva (`FF_S0` a `FF_S5`) e cada termo das equações é uma porta lógica primitiva do Verilog (`and`, `or`, `not`, `buf`) com o mesmo nome do diagrama lógico. Assim há uma correspondência unívoca entre cada *flip-flop*, cada estado do fluxograma e cada linha do código. A saída `Estado` liga as saídas Q aos LEDs LD10 a LD15.

Listagem 11: Controlador: máquina de estados por realização direta, no modelo estrutural (`controlador/controlador.v`)

```
1  `timescale 1ns / 1ps
2
3  // Controlador: máquina de estados síncrona por realização direta (um flip-flop por estado),
4  // descrita no modelo estrutural a partir do fluxograma (aula 1):
5  //
6  //   S0 INICIA       ClrMem, InitLFSR, EnLFSR, RstTemp                    -> S1
7  //   S1 ESPERA_START Ready, EnLFSR             Start = 1 -> S2,  senão fica em S1
8  //   S2 LIBERTA      Ready, EnLFSR             Start = 0 -> S3,  senão fica em S2
9  //   S3 DESLOCA      ShiftMem, TglBeat, StartTemp, EnLFSR                 -> S4
10 //   S4 ESPERA_TEMPO Waiting, EnLFSR           Tempo = 0 -> S5,  senão fica em S4
11 //   S5 ACELERA      Faster, EnLFSR            (opcional)                 -> S3
12 //
13 // Flip-flop FF_Sk = estado Sk: só um está a 1 em cada instante (codificação one-hot).
```

```
// Reset (tecla) põe a máquina em S0 de forma assíncrona: PRE do FF_S0 e CLR dos restantes.
// Os outros elementos da arquitetura são inicializados pelas saídas do estado S0.
// As decisões são produtos lógicos (AND) do estado com a condição testada (desmultiplexer)
// e a reunião de percursos que entram no mesmo estado é uma soma lógica (OR).
// Todas as saídas são de Moore: soma lógica dos estados em que estão ativas.
module Controlador(
    input Clock,
    input Reset,                    // assíncrono, ativo a High
    input Start,                    // tecla Start já sincronizada (1 = premida)
    input Tempo,                    // 1 enquanto o temporizador variável conta
    output ClrMem,                  // limpa a memória e o LED Beat
    output InitLFSR,                // carrega a semente no LFSR
    output EnLFSR,                  // habilita o LFSR (avança livremente)
    output ShiftMem,                // desloca a memória e carrega a corda nova
    output RstTemp,                 // inicializa o temporizador variável
    output StartTemp,               // começa nova contagem do temporizador
    output Faster,                  // encurta o período do temporizador
    output TglBeat,                 // troca o estado do LED Beat
    output Ready,                   // LED Ready
    output Waiting,                 // LED Waiting
    output [5:0] Estado             // Q dos 6 flip-flops (LEDs de depuração)
    );

//   Estados (saídas Q dos flip-flops) e respetivas entradas D (estado seguinte)
     wire S0, S1, S2, S3, S4, S5;
     wire D0, D1, D2, D3, D4, D5;

//   Condições negadas e percursos de saída das decisões
     wire nStart, nTempo;
     wire S1_nStart, S1_Start;      // decisão de S1 (Start = 1 ?)
     wire S2_Start, S2_nStart;      // decisão de S2 (Start = 0 ?)
     wire S4_Tempo, S4_nTempo;      // decisão de S4 (Tempo = 1 ?)

//   ------------------------------------------------------------------ flip-flops (um por estado)
     FDPE #(.INIT(1'b1)) FF_S0 (.Q(S0), .C(Clock), .CE(1'b1), .PRE(Reset), .D(D0));
     FDCE #(.INIT(1'b0)) FF_S1 (.Q(S1), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D1));
     FDCE #(.INIT(1'b0)) FF_S2 (.Q(S2), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D2));
     FDCE #(.INIT(1'b0)) FF_S3 (.Q(S3), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D3));
     FDCE #(.INIT(1'b0)) FF_S4 (.Q(S4), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D4));
     FDCE #(.INIT(1'b0)) FF_S5 (.Q(S5), .C(Clock), .CE(1'b1), .CLR(Reset), .D(D5));

//   ------------------------------------------------------------------ decisões
     not inv_start (nStart, Start);
     not inv_tempo (nTempo, Tempo);

     and dec_s1_0 (S1_nStart, S1, nStart);  // S1 com Start = 0: fica em S1
     and dec_s1_1 (S1_Start,  S1, Start);   // S1 com Start = 1: vai para S2
     and dec_s2_1 (S2_Start,  S2, Start);   // S2 com Start = 1: fica em S2
     and dec_s2_0 (S2_nStart, S2, nStart);  // S2 com Start = 0: vai para S3
     and dec_s4_1 (S4_Tempo,  S4, Tempo);   // S4 com Tempo = 1: fica em S4
     and dec_s4_0 (S4_nTempo, S4, nTempo);  // S4 com Tempo = 0: vai para S5

//   ------------------------------------------------------------------ estado seguinte
     buf  ns0 (D0, 1'b0);                   // nenhum percurso entra em S0 (só a tecla Reset)
     or   ns1 (D1, S0, S1_nStart);          // D1 = S0 + S1./Start
     or   ns2 (D2, S1_Start, S2_Start);     // D2 = S1.Start + S2.Start
     or   ns3 (D3, S2_nStart, S5);          // D3 = S2./Start + S5
     or   ns4 (D4, S3, S4_Tempo);           // D4 = S3 + S4.Tempo
     buf  ns5 (D5, S4_nTempo);              // D5 = S4./Tempo

//   ------------------------------------------------------------------ saídas (Moore)
     buf  o_clrmem   (ClrMem,    S0);
```

```
76      buf  o_initlfsr (InitLFSR,  S0);
77      buf  o_rsttemp  (RstTemp,   S0);
78      or   o_enlfsr   (EnLFSR,    S0, S1, S2, S3, S4, S5);
79      or   o_ready    (Ready,     S1, S2);
80      buf  o_shiftmem (ShiftMem,  S3);
81      buf  o_tglbeat  (TglBeat,   S3);
82      buf  o_starttemp(StartTemp, S3);
83      buf  o_waiting  (Waiting,   S4);
84      buf  o_faster   (Faster,    S5);
85
86      assign Estado = {S5, S4, S3, S2, S1, S0};
87
88  endmodule
```

O esquema RTL gerado pelo Vivado a partir do código (RTL Analysis, Open Elaborated Design, Figura 13) mostra a mesma estrutura: FF_S0 como FDPE, FF_S1 a FF_S5 como FDCE e as portas com os nomes do código (`dec_s1_0`, `ns1`, `o_ready`, …). O Vivado apenas decompõe a OR de 6 entradas de EnLFSR em OR de 2 entradas e junta o inversor de Tempo à AND `dec_s4_0`.

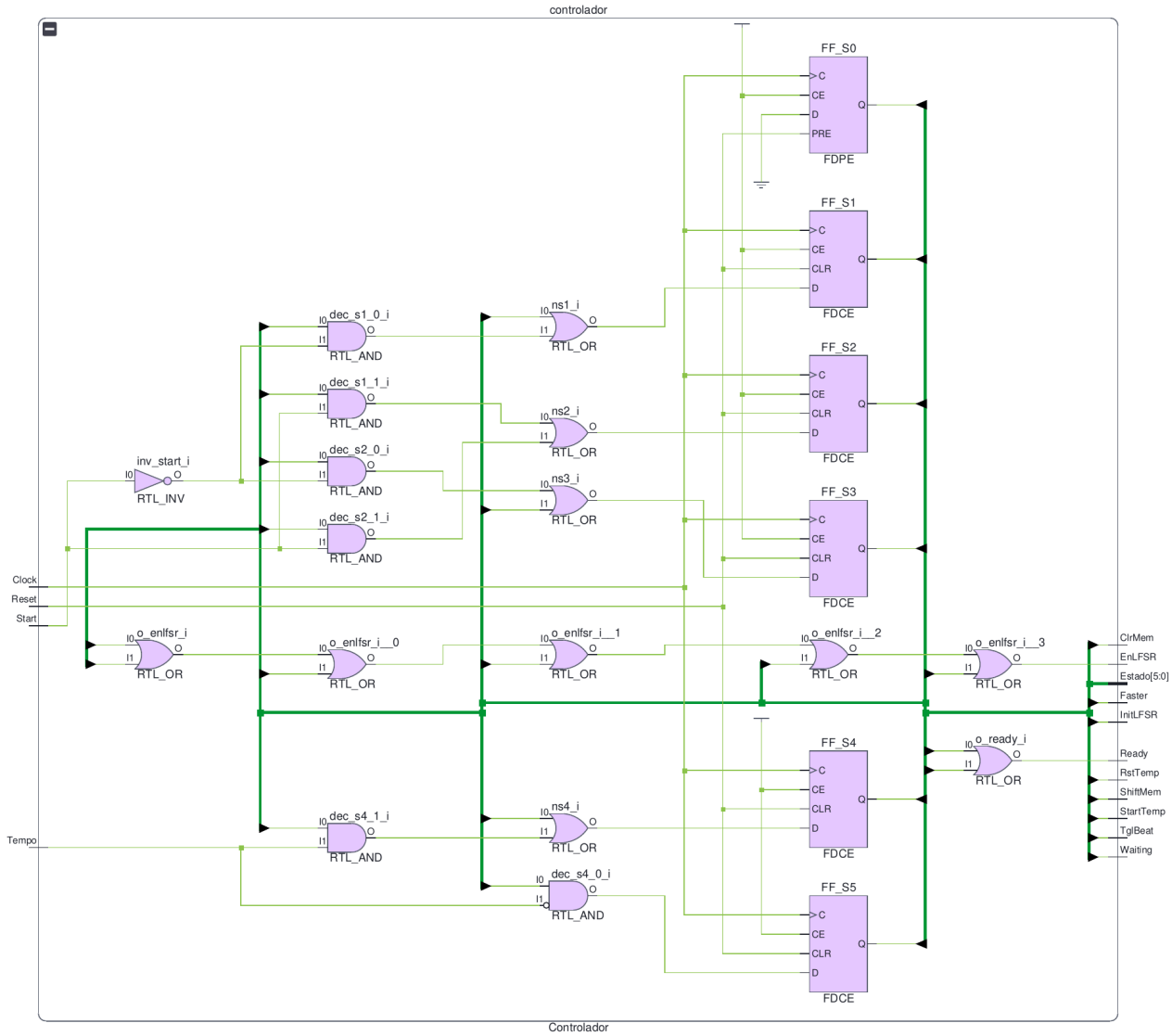


Figura 13: Esquema RTL do controlador gerado pelo Vivado (Elaborated Design).

9.9 Verificação do controlador (passo 11)

O *testbench* do controlador (Listagem 22, no Anexo A) percorre todas as transições do fluxograma: em cada linha muda Start e Tempo, espera um ciclo e confere o estado (os seis *flip-flops*) e as dez saídas de Moore desse estado, escritas numa tabela no próprio *testbench* (as da Tabela 5). O percurso inclui valores das entradas que não interessam em cada estado (Tempo em S1 e S2, Start em S4) e um Reset a meio do ciclo do relógio. Resultado: `Simulacao OK (15 verificacoes)`. Para confirmar que o *testbench* deteta erros, introduziram-se erros propositados no controlador (uma transição trocada, uma saída no estado errado, Tempo em vez de /Tempo na equação de D_S5); a simulação falhou em todos os casos.

Na Figura 14, a máquina está em S0 durante as configurações iniciais (FF_S0 com INIT = 1), com ClrMem, InitLFSR e RstTemp, e passa a S1 no primeiro flanco (105 ns). Fica em S1 enquanto Start = 0, vai para S2 com Start = 1 e para S3 quando Start volta a 0. S3 e S5 duram um ciclo (ShiftMem, StartTemp e TglBeat em S3, Faster em S5) e S4 mantém-se enquanto Tempo = 1. O Reset aos 210 ns põe a máquina em S0 de imediato, sem esperar pelo flanco; largado aos 220 ns, a máquina passa a S1 no flanco seguinte.

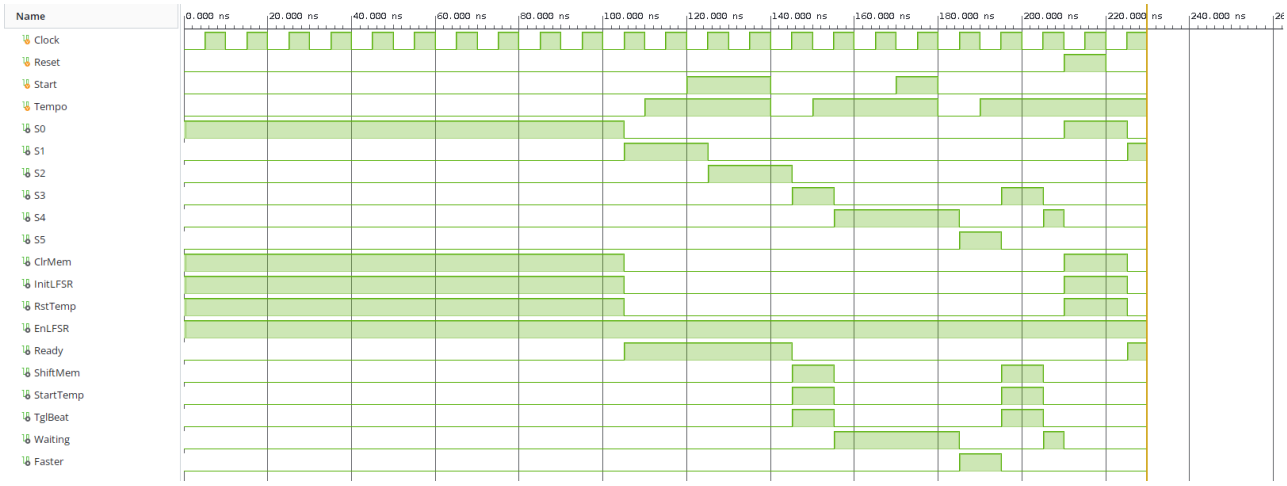


Figura 14: Simulação do controlador: todas as transições do fluxograma e Reset assíncrono.

10 Integração do sistema

10.1 Circuito de topo

O `GuitarHero` (Listagem 12) liga o controlador à arquitetura a controlar, conforme a Figura 1: as ligações são feitas pelo nome do sinal (ClrMem, ShiftMem, Tempo, …). Os tempos longos são parâmetros (`METADE` do varrimento, `ESTAVEL` do anti-ressalto e `INICIAL`, `PASSO` e `MINIMO` do temporizador), com os valores reais por omissão e valores curtos no *testbench*.

Listagem 12: Circuito de topo: controlador e arquitetura a controlar (`guitarhero/guitarhero.v`)

```
`timescale 1ns / 1ps

// Gestão dos displays para o jogo Guitar Hero (2.º trabalho): circuito de topo.
// Liga o controlador (máquina de estados) à arquitetura a controlar:
//   - SincronizadorReset: Reset ativado de imediato e libertado em sincronismo com o relógio;
//   - Sincronizador: tecla Start sincronizada e sem ressaltos;
//   - LFSR32: sequência pseudo-aleatória das cordas;
//   - Memoria4x2: cordas dos 4 displays (fila);
//   - GestaoDisplays: varrimento dos displays com o transcodificador das cordas;
//   - TemporizadorVariavel: velocidade do jogo;
//   - Beat: flip-flop T do LED Beat.
// As ligações entre o controlador e a arquitetura são feitas pelo nome do sinal.
// Os parâmetros permitem usar tempos curtos na simulação.
module GuitarHero #(
    parameter METADE  = 25000,      // varrimento dos displays a 2 kHz
    parameter ESTAVEL = 1000000,    // anti-ressalto da tecla Start: 10 ms
    parameter INICIAL = 50000000,   // período inicial do temporizador: 0,5 s
    parameter PASSO   = 500000,     // aceleração por passo: 5 ms
    parameter MINIMO  = 20000000    // período mínimo: 0,2 s
    )(
    input CLKBasys,
    input BtnReset,
    input BtnStart,
    output [6:0] Seg,
    output DP,
    output [3:0] An,
    output LedReady,
    output LedBeat,
    output LedWaiting,
    output [5:0] LedEstado
    );

//  Sinais entre o controlador e a arquitetura a controlar
     wire Reset, Start, Tempo;                              // entradas do controlador
     wire ClrMem, InitLFSR, EnLFSR, ShiftMem;               // saídas do controlador
     wire RstTemp, StartTemp, Faster, TglBeat;

//  Sinais internos da arquitetura
     wire [1:0] OutLFSR;
     wire [1:0] Pos0, Pos1, Pos2, Pos3;

//  ------------------------------------------------------------------ controlador
     SincronizadorReset sincronizadorReset (
         .Clock(CLKBasys),
         .Entrada(BtnReset),
         .Saida(Reset)
     );

     Controlador controlador (
         .Clock(CLKBasys),
         .Reset(Reset),
```

```
52          .Start(Start),
53          .Tempo(Tempo),
54          .ClrMem(ClrMem),
55          .InitLFSR(InitLFSR),
56          .EnLFSR(EnLFSR),
57          .ShiftMem(ShiftMem),
58          .RstTemp(RstTemp),
59          .StartTemp(StartTemp),
60          .Faster(Faster),
61          .TglBeat(TglBeat),
62          .Ready(LedReady),
63          .Waiting(LedWaiting),
64          .Estado(LedEstado)
65      );
66
67  //  ------------------------------------------------------------------ arquitetura a controlar
68      Sincronizador #(.ESTAVEL(ESTAVEL)) sincronizador (
69          .Clock(CLKBasys),
70          .Entrada(BtnStart),
71          .Saida(Start)
72      );
73
74      LFSR32 lfsr (
75          .Clock(CLKBasys),
76          .Enable(EnLFSR),
77          .Init(InitLFSR),
78          .OutLFSR(OutLFSR)
79      );
80
81      Memoria4x2 memoria (
82          .Data(OutLFSR),
83          .Shift(ShiftMem),
84          .Clk(CLKBasys),
85          .CLR(ClrMem),
86          .Pos0(Pos0),
87          .Pos1(Pos1),
88          .Pos2(Pos2),
89          .Pos3(Pos3)
90      );
91
92      GestaoDisplays #(.METADE(METADE)) displays (
93          .Pos0(Pos0),
94          .Pos1(Pos1),
95          .Pos2(Pos2),
96          .Pos3(Pos3),
97          .Clock(CLKBasys),
98          .Segmentos(Seg),
99          .DotPoint(DP),
100         .Anodos(An)
101     );
102
103     TemporizadorVariavel #(.INICIAL(INICIAL), .PASSO(PASSO), .MINIMO(MINIMO)) temporizador (
104         .Clock(CLKBasys),
105         .RstTemp(RstTemp),
106         .StartTemp(StartTemp),
107         .Faster(Faster),
108         .Tempo(Tempo)
109     );
110
111     Beat beat (
112         .Clk(CLKBasys),
113         .Tgl(TglBeat),
```

```
114          .CLR(ClrMem),
115          .Q(LedBeat)
116      );
117
118  endmodule
```

10.2 Restrições e implementação

A Tabela 12 mostra a ligação dos sinais à Basys3 (Listagem 13). A tecla Start é a tecla central (BTNC) e o Reset a tecla de cima (BTNU).

Tabela 12: Ligação dos sinais do circuito de topo à Basys3.

Sinal	Pino(s)	Elemento da Basys3
CLKBasys	W5	Oscilador de 100 MHz
BtnReset	T18	Tecla BTNU
BtnStart	U18	Tecla BTNC
LedReady / LedBeat / LedWaiting	U16 / E19 / U19	LEDs LD0 / LD1 / LD2
LedEstado[0] a LedEstado[5]	W3 U3 P3 N3 P1 L1	LEDs LD10 a LD15 (S0 a S5)
Seg[6:0]	W7 W6 U8 V8 U5 V5 U7	Segmentos CA a CG (ativos a *Low*)
DP	V7	Ponto decimal (ativo a *Low*)
An[3:0]	W4 V4 U4 U2	Ânodos AN3 a AN0 (ativos a *Low*)

Como no 1.º trabalho, as restrições definem o relógio de 100 MHz e o relógio do varrimento gerado pelo `DivisorClock` (divisão por 50 000), e o caminho de realimentação do divisor entre os dois relógios é declarado como falso, para evitar o aviso crítico TIMING-8. As teclas (que passam pelos sincronizadores), os LEDs e os *displays* não têm requisitos temporais. A implementação cumpre todas as restrições, com uma folga (WNS) de 5,085 ns, e ocupa 97 LUT e 158 *flip-flops*. O relatório de travessias entre domínios de relógio (`report_cdc`) não tem avisos críticos: os únicos sinais assíncronos, as teclas, chegam aos sincronizadores com `ASYNC_REG`.

Listagem 13: Ficheiro de restrições do circuito de topo (`guitarhero/guitarhero.xdc`)

```
1  ## Restrições do circuito GuitarHero na placa Basys3 (xc7a35ticpg236-1L)
2  ## Tecla Reset = BTNU, tecla Start = BTNC
3  ## LD0 = Ready, LD1 = Beat, LD2 = Waiting, LD10 a LD15 = estados S0 a S5 (um LED por flip-flop)
4
5  ## Relógio de 100 MHz
6  set_property PACKAGE_PIN W5 [get_ports CLKBasys]
7      set_property IOSTANDARD LVCMOS33 [get_ports CLKBasys]
8  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLKBasys]
9
10 ## Relógio do varrimento dos displays (100 MHz / 50000 = 2 kHz), gerado pelo DivisorClock
11 ## (-divide_by = 2 x METADE: atualizar se METADE mudar).
12 create_generated_clock -name clk_varrimento -source [get_ports CLKBasys] -divide_by 50000 [get_pins
   displays/divisor/ClockOut_reg/Q]
13 ## O único caminho clk_varrimento -> sys_clk_pin é a realimentação do próprio divisor (ClockOut ->
   /ClockOut,
14 ## um LUT, ~1,5 ns). Sem este false path o Vivado tenta analisá-lo e, como 50000 períodos não têm período
   comum
15 ## dentro do limite que expande, dá TIMING-8 (Critical Warning); com ele fica só TIMING-9, como no 1.º
   trabalho.
16 set_false_path -from [get_clocks clk_varrimento] -to [get_clocks sys_clk_pin]
17
18 ## Teclas (assíncronas: as duas passam por sincronizadores; o Reset atua de imediato no PRE
19 ## dos flip-flops do SincronizadorReset e a libertação para o controlador já é síncrona)
20 set_false_path -from [get_ports {BtnReset BtnStart}]
21 ## LEDs e displays: sem requisitos de tempo
```

```
set_false_path -to [get_ports {Seg[*] DP An[*] LedReady LedBeat LedWaiting LedEstado[*]}]

## Teclas
set_property PACKAGE_PIN T18 [get_ports BtnReset]
    set_property IOSTANDARD LVCMOS33 [get_ports BtnReset]
set_property PACKAGE_PIN U18 [get_ports BtnStart]
    set_property IOSTANDARD LVCMOS33 [get_ports BtnStart]

## LEDs de sinalização
set_property PACKAGE_PIN U16 [get_ports LedReady]
    set_property IOSTANDARD LVCMOS33 [get_ports LedReady]
set_property PACKAGE_PIN E19 [get_ports LedBeat]
    set_property IOSTANDARD LVCMOS33 [get_ports LedBeat]
set_property PACKAGE_PIN U19 [get_ports LedWaiting]
    set_property IOSTANDARD LVCMOS33 [get_ports LedWaiting]

## LEDs dos estados: LD10 = S0 ... LD15 = S5
set_property PACKAGE_PIN W3 [get_ports {LedEstado[0]}]
set_property PACKAGE_PIN U3 [get_ports {LedEstado[1]}]
set_property PACKAGE_PIN P3 [get_ports {LedEstado[2]}]
set_property PACKAGE_PIN N3 [get_ports {LedEstado[3]}]
set_property PACKAGE_PIN P1 [get_ports {LedEstado[4]}]
set_property PACKAGE_PIN L1 [get_ports {LedEstado[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {LedEstado[*]}]

## Displays de 7 segmentos (Seg[6] = a ... Seg[0] = g, ativos a Low)
set_property PACKAGE_PIN W7 [get_ports {Seg[6]}]
set_property PACKAGE_PIN W6 [get_ports {Seg[5]}]
set_property PACKAGE_PIN U8 [get_ports {Seg[4]}]
set_property PACKAGE_PIN V8 [get_ports {Seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {Seg[2]}]
set_property PACKAGE_PIN V5 [get_ports {Seg[1]}]
set_property PACKAGE_PIN U7 [get_ports {Seg[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {Seg[*]}]
set_property PACKAGE_PIN V7 [get_ports DP]
    set_property IOSTANDARD LVCMOS33 [get_ports DP]
set_property PACKAGE_PIN U2 [get_ports {An[0]}]
set_property PACKAGE_PIN U4 [get_ports {An[1]}]
set_property PACKAGE_PIN V4 [get_ports {An[2]}]
set_property PACKAGE_PIN W4 [get_ports {An[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {An[*]}]

## Configuração
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

10.3 Simulação do sistema completo

O *testbench* do sistema (Listagem 23, no Anexo A) usa `METADE = 2`, `ESTAVEL = 4` e um temporizador com `INICIAL = 40`, `PASSO = 4` e `MINIMO = 20` ciclos. O percurso é o de um jogo: a espera em S1, a tecla Start com um ressalto, sete passos do jogo e um Reset a meio do jogo. Em cada passo confere:

- o estado e os LEDs Ready, Waiting e Beat (que troca em cada passo);
- as cordas nas quatro posições da memória: a corda nova entra em `Pos0` e as outras avançam uma posição (as cordas são as que o LFSR dá nesta simulação);
- o período do temporizador, que diminui 4 ciclos por passo, de 40 até ao mínimo de 20;
- no 1.º passo, os segmentos do *display* da esquerda, que mostra `Pos0`.

Depois do Reset confere que a máquina fica em S0 de imediato, com a memória limpa e o LED Beat apagado, e que volta a S1 com o período inicial. Resultado: `Simulacao OK (30 verificacoes).`

Na Figura 15 vê-se o jogo completo: depois de libertado o Reset das configurações iniciais, Ready acende (S1); `Start` (já sem o ressalto) leva a máquina a S2 e, quando é largada, o jogo começa. As cordas entram em `Pos0` e avançam até `Pos3`, `Periodo` diminui de 40 para 20 ciclos e o LED Beat troca a cada passo. O Reset a meio do jogo limpa a memória e o LED Beat e repõe o período de 40 ciclos.

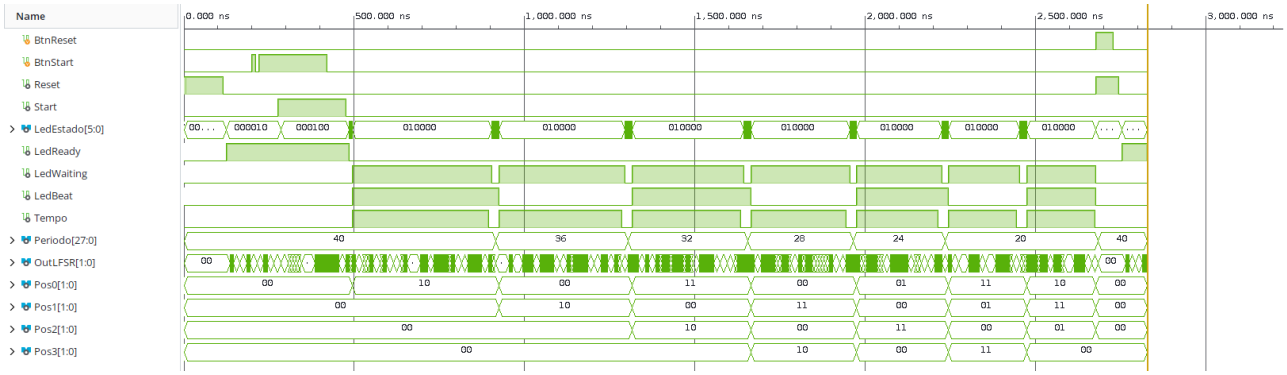


Figura 15: Simulação do sistema completo: Start, sete passos do jogo e Reset a meio do jogo.

A Figura 16 mostra o primeiro passo em pormenor. Quando `Start` volta a 0, a máquina passa de S2 a S3: nesse ciclo ShiftMem carrega a corda do LFSR (10) em `Pos0` e StartTemp inicia o temporizador, que mantém `Tempo` a 1 durante 40 ciclos (S4). Quando `Tempo` volta a 0, a máquina passa por S5 (Faster: o período passa a 36) e regressa a S3, onde a corda seguinte (00, pausa) entra em `Pos0` e o 10 passa para `Pos1`. Entre os dois passos decorrem 43 ciclos (40 + 3).

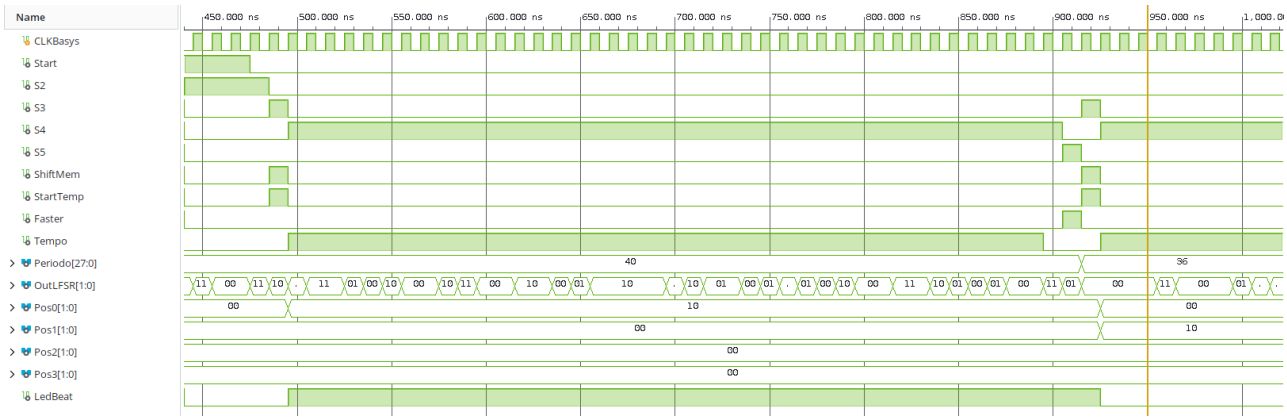


Figura 16: Pormenor da simulação do sistema: dois passos do jogo (S3, S4, S5 e S3).

10.4 Integração faseada e ensaio na placa

A integração foi feita módulo a módulo, como pede o enunciado: cada módulo foi simulado isoladamente (secções 3 a 8), depois o controlador sozinho (secção 9.9) e, por fim, o sistema completo (secção 10.3). Na placa, o ensaio segue a ordem da Tabela 13; os LEDs LD10 a LD15 mostram em cada instante o *flip-flop* ativo, o que permite seguir o fluxograma na própria placa.

Tabela 13: Procedimento de ensaio na Basys3 e resultado esperado.

Passo	Ação	Resultado esperado
1	Programar a FPGA com guitarhero.bit	Displays apagados; LD11 (S1) e LD0 (Ready) acesos
2	Carregar BTNC (Start) e mantê-la carregada	LD12 (S2) aceso; Ready continua aceso; displays apagados
3	Largar BTNC	Ready apaga; LD14 (S4) e LD2 (Waiting) acesos; as cordas entram pelo *display* da esquerda e avançam para a direita a cada 0,5 s; LD1 (Beat) troca a cada passo
4	Esperar cerca de 20 s	A sequência acelera até 0,2 s por passo e mantém essa velocidade
5	Carregar BTNU (Reset) a meio do jogo	Displays e LD1 apagam de imediato; LD10 (S0) aceso enquanto a tecla está carregada
6	Largar BTNU e repetir os passos 2 e 3	Novo jogo, outra vez a 0,5 s por passo e com outra sequência de cordas

[A COMPLETAR NO LABORATÓRIO: confirmar o resultado de cada passo da Tabela 13 e substituir as fotografias abaixo]

[A INSERIR NO LABORATÓRIO: fotografia da placa depois do Reset (passo 1)]

Figura 17: Ensaio na placa Basys3 depois do Reset: LED Ready e LD11 (S1) acesos, displays apagados.

[A INSERIR NO LABORATÓRIO: fotografia da placa durante o jogo (passo 3)]

Figura 18: Ensaio na placa Basys3 durante o jogo: cordas nos quatro displays, LEDs Waiting e LD14 (S4) acesos.

11 Análise crítica do código de fontes externas

11.1 LFSR do enunciado

O `LFSR32` do enunciado funciona e as derivações estão certas (32, 22, 2 e 1 na XAPP052 para 32 *bits*), mas tem alguns pormenores a notar:

- o comentário pede uma semente "<> 0", o que é válido para um LFSR com XOR; com XNOR o estado proibido é tudo a 1 e o 0 é um estado válido (é, aliás, o valor do registo depois de configurar a FPGA). A semente `32'h03E3180A` serve nos dois casos;
- `Init` só atua com `Enable` a 1, porque está dentro do `if (Enable)`: o estado inicial da máquina tem de ativar InitLFSR e EnLFSR ao mesmo tempo;
- o valor inicial de `regXNOR` (`reg regXNOR = 1`) não tem efeito, porque é um sinal combinatório atribuído num `always @(*)`;
- `OutLFSR` usa os *bits* 31 e 0, que em ciclos consecutivos estão correlacionados (o LFSR desloca um *bit* por ciclo); como a memória só lê um par de *bits* a cada `Periodo` + 3 ciclos, os pares lidos são praticamente independentes, o que a simulação confirmou (cerca de 25 % para cada corda).

O código foi usado sem alterações, só com comentários.

11.2 Módulos do 1.º trabalho

O `Registo4`, o `DivisorClock`, o `Contador4`, o `multiplexer16x4` e o `descodificador2x4` foram reutilizados sem alterações: os projetos e os *scripts* do 2.º trabalho referem os ficheiros da pasta do 1.º trabalho, sem os copiar, pelo que uma correção feita num módulo passa a valer nos dois trabalhos. O transcodificador foi substituído por uma versão nova (`TranscodCordas`), porque a função é diferente (um segmento horizontal por corda).

11.3 Primitivas FDPE e FDCE

As primitivas seguem os modelos do Vivado: `PRE` e `CLR` são assíncronos e ativos a *High*, `CE` é ativo a *High* e `INIT` define o valor depois de configurar a FPGA. FF_S0 tem `INIT = 1` e os outros `INIT = 0`, pelo que a máquina arranca em S0 mesmo sem carregar Reset; o sincronizador do Reset também tem `INIT = 1`, mantendo o Reset ativo durante os dois primeiros ciclos depois da configuração.

11.4 IA generativa

Foi usada IA generativa como apoio na escrita dos *testbenches*, na revisão do código Verilog e dos comentários, na preparação dos diagramas no draw.io e na correção da redação deste relatório, bem como para confirmar que o relatório inclui os elementos pedidos no enunciado. Todo o código e todos os diagramas foram revistos pelo grupo e as simulações foram corridas e analisadas pelo grupo.

12 Conclusão

Desenvolveu-se o sistema de gestão dos *displays* do jogo Guitar Hero: um LFSR de 32 *bits* gera a sequência de cordas e pausas, uma memória 4x2 guarda o que cada *display* mostra e um temporizador variável define e acelera o ritmo do jogo. A gestão dos *displays* reutilizou o sistema do 1.º trabalho, com um novo transcodificador.

O controlador, o cerne do trabalho, foi desenhado a partir do fluxograma pelos 11 passos da metodologia e realizado de forma direta, com um *flip-flop* por estado, no modelo estrutural: cada *flip-flop* corresponde a um estado do fluxograma e cada porta a um termo das equações, o que o esquema RTL do Vivado confirmou. O trabalho mostrou também a importância da sincronização das entradas assíncronas: a tecla Start precisa de sincronizador e anti-ressalto, e a libertação do Reset tem de ser síncrona para evitar uma corrida crítica que deixaria a máquina sem estado ativo.

Os 10 *testbenches* (um por módulo, o do controlador e o do sistema completo) conferem as saídas com os valores esperados e terminam todos com `Simulacao OK`; a implementação cumpre as restrições temporais.

[A COMPLETAR NO LABORATÓRIO: acrescentar o resultado do ensaio na placa Basys3 (secção 10.4)]

Anexo A Testbenches

Listagens dos *testbenches* das secções 3 a 10. Todos verificam as saídas automaticamente e terminam com `Simulacao OK` ou `Simulacao FALHOU`; as vistas das formas de onda usadas nas figuras estão nos ficheiros `.wcfg` ao lado de cada *testbench*.

Listagem 14: Testbench do sincronizador da tecla Start (`sincronizador/sincronizador_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do sincronizador / anti-ressalto da tecla Start.
// Com ESTAVEL = 4 ciclos (na placa são 1 000 000 = 10 ms): a saída só muda depois de a
// entrada ficar 4 ciclos estável, mais 2 ciclos dos flip-flops de sincronização.
module sincronizador_teste1();

//   Declaração das entradas
     reg Clock = 0;
     reg Entrada = 0;

//   Declaração da saída
     wire Saida;

//   Instanciação da UUT e ligação das entradas e saídas
     Sincronizador #(.ESTAVEL(4)) UUT (
         .Clock(Clock),
         .Entrada(Entrada),
         .Saida(Saida)
     );

//   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
     always #5 Clock = ~Clock;

//   Confere a saída com o valor esperado
     integer erros = 0, verificacoes = 0;
     task confere(input esperado);
         begin
             verificacoes = verificacoes + 1;
             if (Saida !== esperado) begin
                 $display("ERRO t=%0t: Saida=%b (esperado %b)", $time, Saida, esperado);
                 erros = erros + 1;
             end
         end
     endtask

     initial begin
         #10;                                    // as entradas mudam a meio do ciclo
         // Ruído: impulso de 2 ciclos, mais curto do que ESTAVEL
         Entrada = 1; #20 Entrada = 0;
         #100 confere(0);                        // ignorado

         // Carregar a tecla com ressaltos
         Entrada = 1; #20 Entrada = 0; #10 Entrada = 1; #20 Entrada = 0; #10 Entrada = 1;
         #50 confere(0);                         // 5 ciclos depois do último ressalto: ainda não
         #20 confere(1);                         // 7 ciclos depois: Saida = 1 (uma só subida)
         #100 confere(1);

         // Largar a tecla com ressaltos
         Entrada = 0; #20 Entrada = 1; #10 Entrada = 0;
         #50 confere(1);                         // ainda não
         #20 confere(0);                         // Saida = 0 (uma só descida)
         #100 confere(0);

```

```
55          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
56          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
57          $finish;
58      end
59
60  endmodule
```

Listagem 15: Testbench do sincronizador do Reset (`sincronizador/sincronizadorreset_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do sincronizador do Reset.
4   // A saída sobe logo que a tecla é carregada (sem esperar pelo relógio) e só desce no
5   // 2.º flanco ascendente depois de a tecla ser largada.
6   module sincronizadorreset_teste1();
7
8   //   Declaração das entradas
9       reg Clock = 0;
10      reg Entrada = 0;
11
12  //   Declaração da saída
13      wire Saida;
14
15  //   Instanciação da UUT e ligação das entradas e saídas
16      SincronizadorReset UUT (
17          .Clock(Clock),
18          .Entrada(Entrada),
19          .Saida(Saida)
20      );
21
22  //   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
23      always #5 Clock = ~Clock;
24
25  //   Confere a saída com o valor esperado
26      integer erros = 0, verificacoes = 0;
27      task confere(input esperado);
28          begin
29              verificacoes = verificacoes + 1;
30              if (Saida !== esperado) begin
31                  $display("ERRO t=%0t: Saida=%b (esperado %b)", $time, Saida, esperado);
32                  erros = erros + 1;
33              end
34          end
35      endtask
36
37      initial begin
38          // Depois de configurar a FPGA (INIT = 1) o Reset está ativo
39          #50  confere(1);
40          #60  confere(1);           // t = 110: fim das configurações (100 ns), 1.º flanco em 105
41          #10  confere(0);           // t = 120: 2.º flanco em 115, Reset libertado
42
43          // Carregar a tecla entre dois flancos: sobe logo
44          #12  Entrada = 1;          // t = 132
45          #1   confere(1);           // t = 133, antes do flanco de 135
46          #20  Entrada = 0;          // t = 153: largar
47          #5   confere(1);           // t = 158: 1.º flanco (155), ainda ativo
48          #10  confere(0);           // t = 168: 2.º flanco (165), libertado
49
50          // Impulso de 2 ns entre dois flancos: também ativa o Reset
51          #14  Entrada = 1;          // t = 182
52          #1   confere(1);
```

```
        #1   Entrada = 0;          // t = 184
        #4   confere(1);           // t = 188: 1.º flanco (185)
        #10  confere(0);           // t = 198: 2.º flanco (195)

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 16: Testbench do LFSR de 32 bits (`lfsr32/lfsr32_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do LFSR de 32 bits.
// Confere o registo interno (regLFSR) e a saída OutLFSR = {regLFSR[31], regLFSR[0]}
// com os valores calculados à mão: em cada passo o registo desloca para a esquerda e entra
// à direita a XNOR dos bits 31, 21, 1 e 0.
module lfsr32_teste1();

//   Declaração das entradas
    reg Clock = 0;
    reg Enable = 0, Init = 0;

//   Declaração da saída
    wire [1:0] OutLFSR;

//   Instanciação da UUT e ligação das entradas e saídas
    LFSR32 #(.SEED(32'h03E3180A)) UUT (
        .Clock(Clock),
        .Enable(Enable),
        .Init(Init),
        .OutLFSR(OutLFSR)
    );

//   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
    always #5 Clock = ~Clock;

//   Confere o registo e a saída com os valores esperados
    integer erros = 0, verificacoes = 0;
    task confere(input [31:0] registo, input [1:0] saida);
        begin
            verificacoes = verificacoes + 1;
            if (UUT.regLFSR !== registo || OutLFSR !== saida) begin
                $display("ERRO t=%0t: regLFSR=%h OutLFSR=%b (esperado %h %b)",
                         $time, UUT.regLFSR, OutLFSR, registo, saida);
                erros = erros + 1;
            end
        end
    endtask

    initial begin
        #10;                                          // as entradas mudam a meio do ciclo
                           confere(32'h00000000, 2'b00);   // valor inicial
        Init = 1;      #10 confere(32'h00000000, 2'b00);   // Enable = 0: Init não tem efeito
        Enable = 1;    #10 confere(32'h03E3180A, 2'b00);   // Enable = 1 e Init = 1: carrega a semente
        Init = 0;      #10 confere(32'h07C63015, 2'b01);   // 1.º passo: entra 1 à direita
                       #10 confere(32'h0F8C602A, 2'b00);   // 2.º passo: entra 0
                       #10 confere(32'h1F18C054, 2'b00);   // 3.º passo: entra 0
        Enable = 0;    #30 confere(32'h1F18C054, 2'b00);   // Enable = 0: mantém durante 3 flancos
```

```
49          Enable = 1;    #10 confere(32'h3E3180A9, 2'b01);   // volta a avançar
50                         #10 confere(32'h7C630153, 2'b01);
51
52          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
53          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
54          $finish;
55      end
56
57  endmodule
```

Listagem 17: Testbench da memória 4x2 (`memoria4x2/memoria4x2_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench da memória 4x2 (fila de 4 posições de 2 bits).
4   // Cada linha muda as entradas, espera e confere as 4 posições (Pos0, Pos1, Pos2, Pos3).
5   module memoria4x2_teste1();
6
7   //   Declaração das entradas
8        reg [1:0] Data = 2'b00;
9        reg Shift = 0, CLR = 0;
10       reg Clk = 0;
11
12  //   Declaração das saídas
13       wire [1:0] Pos0, Pos1, Pos2, Pos3;
14
15  //   Instanciação da UUT e ligação das entradas e saídas
16       Memoria4x2 UUT (
17           .Data(Data),
18           .Shift(Shift),
19           .Clk(Clk),
20           .CLR(CLR),
21           .Pos0(Pos0),
22           .Pos1(Pos1),
23           .Pos2(Pos2),
24           .Pos3(Pos3)
25       );
26
27  //   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
28       always #5 Clk = ~Clk;
29
30  //   Confere as 4 posições com os valores esperados
31       integer erros = 0, verificacoes = 0;
32       task confere(input [1:0] p0, input [1:0] p1, input [1:0] p2, input [1:0] p3);
33           begin
34               verificacoes = verificacoes + 1;
35               if ({Pos0, Pos1, Pos2, Pos3} !== {p0, p1, p2, p3}) begin
36                   $display("ERRO t=%0t: Pos0..Pos3 = %b %b %b %b (esperado %b %b %b %b)",
37                            $time, Pos0, Pos1, Pos2, Pos3, p0, p1, p2, p3);
38                   erros = erros + 1;
39               end
40           end
41       endtask
42
43       initial begin
44           #110;                                        // configurações iniciais da FPGA (100 ns)
45                                  confere(2'b00, 2'b00, 2'b00, 2'b00);   // valor inicial: tudo em pausa
46           Shift = 1; Data = 2'b01; #10 confere(2'b01, 2'b00, 2'b00, 2'b00);   // 01 entra em Pos0
47                      Data = 2'b10; #10 confere(2'b10, 2'b01, 2'b00, 2'b00);   // e o 01 passa a Pos1
48                      Data = 2'b11; #10 confere(2'b11, 2'b10, 2'b01, 2'b00);
49                      Data = 2'b00; #10 confere(2'b00, 2'b11, 2'b10, 2'b01);
```

```
                    Data = 2'b10; #10 confere(2'b10, 2'b00, 2'b11, 2'b10);   // o 01 sai de Pos3
        Shift = 0; Data = 2'b01; #30 confere(2'b10, 2'b00, 2'b11, 2'b10);   // Shift = 0: mantém
        CLR = 1;                 #2  confere(2'b00, 2'b00, 2'b00, 2'b00);   // CLR limpa logo, sem flanco
        Shift = 1;               #18 confere(2'b00, 2'b00, 2'b00, 2'b00);   // CLR tem prioridade
        CLR = 0;                 #10 confere(2'b01, 2'b00, 2'b00, 2'b00);   // volta a deslocar

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 18: Testbench do temporizador (`temporizador/temporizador_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do temporizador.
// Com Start a 1 o temporizador carrega Periodo; Tempo fica a 1 durante Periodo ciclos.
// Cada linha muda as entradas, espera e confere Tempo.
module temporizador_teste1();

//   Declaração das entradas
    reg Clock = 0;
    reg Reset = 0, Start = 0;
    reg [7:0] Periodo = 8'd0;

//   Declaração da saída
    wire Tempo;

//   Instanciação da UUT (contador de 8 bits para a simulação)
    Temporizador #(.N(8)) UUT (
        .Clock(Clock),
        .Reset(Reset),
        .Start(Start),
        .Periodo(Periodo),
        .Tempo(Tempo)
    );

//   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
    always #5 Clock = ~Clock;

//   Confere Tempo com o valor esperado
    integer erros = 0, verificacoes = 0;
    task confere(input esperado);
        begin
            verificacoes = verificacoes + 1;
            if (Tempo !== esperado) begin
                $display("ERRO t=%0t: Tempo=%b (esperado %b)", $time, Tempo, esperado);
                erros = erros + 1;
            end
        end
    endtask

    initial begin
        #10;                                        // as entradas mudam a meio do ciclo
                                    confere(0);     // valor inicial: parado
        // Periodo = 3: Tempo a 1 durante 3 ciclos
        Periodo = 3; Start = 1;     #10 confere(1);     // 1.º ciclo
                     Start = 0;     #10 confere(1);     // 2.º ciclo
                                    #10 confere(1);     // 3.º ciclo
```

```
                                    #10 confere(0);     // acabou sozinho
                                    #30 confere(0);
        // Periodo = 0: Tempo não chega a subir
        Periodo = 0; Start = 1;     #10 confere(0);
                     Start = 0;     #10 confere(0);
        // Reset a meio da contagem: para
        Periodo = 5; Start = 1;     #10 confere(1);
                     Start = 0;     #10 confere(1);
        Reset = 1;                  #10 confere(0);     // Reset interrompe a contagem
        Reset = 0;                  #30 confere(0);     // e não volta a contar
        // Reset tem prioridade sobre Start
        Reset = 1; Start = 1;       #10 confere(0);
        Reset = 0; Start = 0;
        // Start a meio da contagem: recomeça com o novo período
        Periodo = 3; Start = 1;     #10 confere(1);
                     Start = 0;     #10 confere(1);
        Periodo = 4; Start = 1;     #10 confere(1);     // Start outra vez: recomeça com 4 ciclos
                     Start = 0;     #20 confere(1);     // sem recomeçar já teria acabado
                                    #20 confere(0);     // acaba 4 ciclos depois do 2.º Start

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 19: Testbench do temporizador variável (`temporizador/temporizadorvariavel_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do temporizador variável (registo da velocidade + temporizador).
// Com tempos curtos (INICIAL = 5, PASSO = 1, MINIMO = 3 ciclos) confere o período guardado
// no registo da velocidade e a saída Tempo:
//   - RstTemp põe o período em 5 ciclos;
//   - cada Faster encurta-o 1 ciclo, até ao mínimo de 3;
//   - StartTemp põe Tempo a 1 durante o período atual.
module temporizadorvariavel_teste1();

//   Declaração das entradas
    reg Clock = 0;
    reg RstTemp = 0, StartTemp = 0, Faster = 0;

//   Declaração da saída
    wire Tempo;

//   Instanciação da UUT e ligação das entradas e saídas
    TemporizadorVariavel #(.N(8), .INICIAL(5), .PASSO(1), .MINIMO(3)) UUT (
        .Clock(Clock),
        .RstTemp(RstTemp),
        .StartTemp(StartTemp),
        .Faster(Faster),
        .Tempo(Tempo)
    );

//   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
    always #5 Clock = ~Clock;

//   Confere o período (registo da velocidade) e a saída Tempo
    integer erros = 0, verificacoes = 0;
    task confere(input [7:0] periodo, input tempo);
```

```
33          begin
34              verificacoes = verificacoes + 1;
35              if (UUT.Periodo !== periodo || Tempo !== tempo) begin
36                  $display("ERRO t=%0t: Periodo=%0d Tempo=%b (esperado %0d %b)",
37                           $time, UUT.Periodo, Tempo, periodo, tempo);
38                  erros = erros + 1;
39              end
40          end
41      endtask
42
43      initial begin
44          #10;                                                // as entradas mudam a meio do ciclo
45          RstTemp = 1;                 #10 confere(5, 0);     // período inicial: 5 ciclos
46          RstTemp = 0; StartTemp = 1;  #10 confere(5, 1);     // Tempo a 1 durante 5 ciclos
47                       StartTemp = 0;  #40 confere(5, 1);     // 5.º ciclo
48                                       #10 confere(5, 0);     // acabou
49          Faster = 1;                  #10 confere(4, 0);     // Faster: 5 - 1 = 4
50          Faster = 0;                  #10 confere(4, 0);     // só muda com Faster
51          Faster = 1;                  #10 confere(3, 0);     // 4 - 1 = 3 (o mínimo)
52                                       #10 confere(3, 0);     // não desce abaixo de 3
53          Faster = 0; StartTemp = 1;   #10 confere(3, 1);     // Tempo a 1 durante 3 ciclos
54                      StartTemp = 0;   #20 confere(3, 1);     // 3.º ciclo
55                                       #10 confere(3, 0);     // acabou
56          StartTemp = 1;               #10 confere(3, 1);     // nova contagem
57          StartTemp = 0; RstTemp = 1;  #10 confere(5, 0);     // RstTemp: para e repõe os 5 ciclos
58          RstTemp = 0;                 #10 confere(5, 0);
59
60          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
61          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
62          $finish;
63      end
64
65  endmodule
```

Listagem 20: Testbench do flip-flop T do LED Beat (`beat/beat_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do flip-flop T do LED Beat.
4   // Cada linha do bloco initial muda uma entrada, espera e confere a saída Q.
5   module beat_teste1();
6
7   //   Declaração das entradas
8        reg Clk = 0;
9        reg Tgl = 0, CLR = 0;
10
11  //   Declaração da saída
12       wire Q;
13
14  //   Instanciação da UUT e ligação das entradas e saídas
15       Beat UUT (
16           .Clk(Clk),
17           .Tgl(Tgl),
18           .CLR(CLR),
19           .Q(Q)
20       );
21
22  //   Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
23       always #5 Clk = ~Clk;
24
25  //   Confere Q com o valor esperado
```

```
26      integer erros = 0, verificacoes = 0;
27      task confere(input esperado);
28          begin
29              verificacoes = verificacoes + 1;
30              if (Q !== esperado) begin
31                  $display("ERRO t=%0t: Q=%b (esperado %b)", $time, Q, esperado);
32                  erros = erros + 1;
33              end
34          end
35      endtask
36
37      initial begin
38          #110;                       // configuração da FPGA (100 ns); entradas a meio do ciclo
39                   confere(0);        // valor inicial (INIT = 0)
40          Tgl = 1; #10 confere(1);    // Tgl = 1: troca em cada flanco
41                   #10 confere(0);
42                   #10 confere(1);
43          Tgl = 0; #30 confere(1);    // Tgl = 0: mantém durante 3 flancos
44          CLR = 1; #2  confere(0);    // CLR apaga logo, sem esperar pelo flanco
45          Tgl = 1; #18 confere(0);    // com CLR a 1 o Tgl não tem efeito
46          CLR = 0; #10 confere(1);    // CLR a 0: volta a trocar
47
48          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
49          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
50          $finish;
51      end
52
53  endmodule
```

Listagem 21: Testbench da gestão dos displays (`gestaodisplays/gestaodisplays_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench da gestão dos displays (com o transcodificador das cordas).
4   // Com o varrimento acelerado (METADE = 2) o display ativo muda a cada 40 ns. Põe uma corda
5   // diferente em cada posição da memória e confere, display a display, o ânodo ativo e os
6   // segmentos acesos (Pos0 no display da esquerda, Pos3 no da direita).
7   module gestaodisplays_teste1();
8
9   //   Declaração das entradas
10      reg [1:0] Pos0 = 0, Pos1 = 0, Pos2 = 0, Pos3 = 0;
11      reg Clock = 0;
12
13  //   Declaração das saídas
14      wire [6:0] Segmentos;
15      wire DotPoint;
16      wire [3:0] Anodos;
17
18  //   Instanciação da UUT (varrimento a 100 MHz / 4 para a simulação ser curta)
19      GestaoDisplays #(.METADE(2)) UUT (
20          .Pos0(Pos0),
21          .Pos1(Pos1),
22          .Pos2(Pos2),
23          .Pos3(Pos3),
24          .Clock(Clock),
25          .Segmentos(Segmentos),
26          .DotPoint(DotPoint),
27          .Anodos(Anodos)
28      );
29
30  //   Relógio de 100 MHz (período de 10ns)
```

```
31      always #5 Clock = ~Clock;
32
33  //  Segmentos (abcdefg, ativos a Low) de cada corda
34      localparam PAUSA = 7'b1111111, CIMA = 7'b0111111, MEIO = 7'b1111110, BAIXO = 7'b1110111;
35
36  //  Confere o display ativo, os segmentos e o ponto decimal (sempre apagado)
37      integer erros = 0, verificacoes = 0;
38      task confere(input [3:0] anodos, input [6:0] segmentos);
39          begin
40              verificacoes = verificacoes + 1;
41              if (Anodos !== anodos || Segmentos !== segmentos || DotPoint !== 1'b1) begin
42                  $display("ERRO t=%0t: Anodos=%b Segmentos=%b DP=%b (esperado %b %b 1)",
43                           $time, Anodos, Segmentos, DotPoint, anodos, segmentos);
44                  erros = erros + 1;
45              end
46          end
47      endtask
48
49      initial begin
50          // Pos0 a Pos3: cima, pausa, baixo, meio
51          Pos0 = 2'b01; Pos1 = 2'b00; Pos2 = 2'b11; Pos3 = 2'b10;
52          #100;                                   // configurações iniciais da FPGA (100 ns)
53          @(negedge Anodos[0]) #20;               // a meio do tempo do display da direita
54              confere(4'b1110, MEIO);             // AN0 (direita): Pos3
55          #40 confere(4'b1101, BAIXO);            // AN1: Pos2
56          #40 confere(4'b1011, PAUSA);            // AN2: Pos1 (display apagado)
57          #40 confere(4'b0111, CIMA);             // AN3 (esquerda): Pos0
58          #40 confere(4'b1110, MEIO);             // volta ao da direita
59
60          // As cordas avançam uma posição (entra a corda de baixo à esquerda)
61          Pos0 = 2'b11; Pos1 = 2'b01; Pos2 = 2'b00; Pos3 = 2'b11;
62          #40 confere(4'b1101, PAUSA);            // AN1: Pos2
63          #40 confere(4'b1011, CIMA);             // AN2: Pos1
64          #40 confere(4'b0111, BAIXO);            // AN3: Pos0
65          #40 confere(4'b1110, BAIXO);            // AN0: Pos3
66
67          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
68          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
69          $finish;
70      end
71
72  endmodule
```

Listagem 22: Testbench do controlador (`controlador/controlador_teste1.v`)

```
1   `timescale 1ns / 1ps
2
3   // Testbench do controlador (máquina de estados por realização direta).
4   // Percorre todas as transições do fluxograma. Em cada linha muda Start/Tempo, espera um ciclo
5   // e confere o estado (os 6 flip-flops) e as 10 saídas de Moore desse estado.
6   module controlador_teste1();
7
8   //  Declaração das entradas
9       reg Clock = 0;
10      reg Reset = 0, Start = 0, Tempo = 0;
11
12  //  Declaração das saídas
13      wire ClrMem, InitLFSR, EnLFSR, ShiftMem, RstTemp, StartTemp, Faster, TglBeat, Ready, Waiting;
14      wire [5:0] Estado;
15
16  //  Instanciação da UUT e ligação das entradas e saídas
```

```
17      Controlador UUT (
18          .Clock(Clock),
19          .Reset(Reset),
20          .Start(Start),
21          .Tempo(Tempo),
22          .ClrMem(ClrMem),
23          .InitLFSR(InitLFSR),
24          .EnLFSR(EnLFSR),
25          .ShiftMem(ShiftMem),
26          .RstTemp(RstTemp),
27          .StartTemp(StartTemp),
28          .Faster(Faster),
29          .TglBeat(TglBeat),
30          .Ready(Ready),
31          .Waiting(Waiting),
32          .Estado(Estado)
33      );
34
35  //  Relógio de 100 MHz (período de 10ns): flancos ascendentes em 5, 15, 25, ... ns
36      always #5 Clock = ~Clock;
37
38  //  Estados one-hot: Estado = {S5, S4, S3, S2, S1, S0}
39      localparam S0 = 6'b000001, S1 = 6'b000010, S2 = 6'b000100,
40                 S3 = 6'b001000, S4 = 6'b010000, S5 = 6'b100000;
41
42  //  Saídas de cada estado, pela ordem
43  //  ClrMem InitLFSR EnLFSR ShiftMem RstTemp StartTemp Faster TglBeat Ready Waiting
44      function [9:0] saidas_de(input [5:0] estado);
45          case (estado)
46              S0: saidas_de = 10'b1_1_1_0_1_0_0_0_0_0;
47              S1: saidas_de = 10'b0_0_1_0_0_0_0_0_1_0;
48              S2: saidas_de = 10'b0_0_1_0_0_0_0_0_1_0;
49              S3: saidas_de = 10'b0_0_1_1_0_1_0_1_0_0;
50              S4: saidas_de = 10'b0_0_1_0_0_0_0_0_0_1;
51              S5: saidas_de = 10'b0_0_1_0_0_0_1_0_0_0;
52          endcase
53      endfunction
54      wire [9:0] saidas = {ClrMem, InitLFSR, EnLFSR, ShiftMem, RstTemp,
55                           StartTemp, Faster, TglBeat, Ready, Waiting};
56
57  //  Confere o estado e as saídas
58      integer erros = 0, verificacoes = 0;
59      task confere(input [5:0] estado);
60          begin
61              verificacoes = verificacoes + 1;
62              if (Estado !== estado || saidas !== saidas_de(estado)) begin
63                  $display("ERRO t=%0t: Estado=%b saidas=%b (esperado %b %b)",
64                           $time, Estado, saidas, estado, saidas_de(estado));
65                  erros = erros + 1;
66              end
67          end
68      endtask
69
70      initial begin
71          #50  confere(S0);                       // configuração da FPGA: S0 (INIT = 1 só em FF_S0)
72          #60  confere(S1);                       // t = 110: no 1.º flanco (105) passa a S1
73          // As entradas mudam a meio do ciclo; cada confere é feito depois de um flanco
74          Tempo = 1;            #10 confere(S1);  // S1 espera Start (Tempo não interessa)
75          Start = 1;            #10 confere(S2);  // Start = 1: S2
76                                #10 confere(S2);  // fica em S2 enquanto Start = 1
77          Start = 0; Tempo = 0; #10 confere(S3);  // Start largada: S3 (1.º passo do jogo)
78          Tempo = 1;            #10 confere(S4);  // S3 -> S4 sempre
```

```
                                #10 confere(S4);  // fica em S4 enquanto Tempo = 1
        Start = 1;              #10 confere(S4);  // Start não interessa em S4
        Start = 0; Tempo = 0; #10 confere(S5);  // fim do tempo: S5
        Tempo = 1;              #10 confere(S3);  // S5 -> S3 sempre (passo seguinte)
                                #10 confere(S4);
        // Reset a meio do ciclo
        Reset = 1;              #1  confere(S0);  // S0 logo, sem esperar pelo flanco
                                #9  confere(S0);  // fica em S0 enquanto Reset = 1
        Reset = 0;              #10 confere(S1);  // Reset largado: S1 no flanco seguinte

        if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
        else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
        $finish;
    end

endmodule
```

Listagem 23: Testbench do sistema completo (`guitarhero/guitarhero_teste1.v`)

```
`timescale 1ns / 1ps

// Testbench do circuito completo (controlador + arquitetura a controlar).
// Tempos curtos: varrimento METADE = 2, anti-ressalto de 4 ciclos e temporizador com
// INICIAL = 40, PASSO = 4, MINIMO = 20 ciclos.
// Percurso: espera em S1, Start com um ressalto, 7 passos do jogo e Reset a meio do jogo.
// Em cada passo confere os LEDs, as cordas da memória (que avançam uma posição) e o período.
module guitarhero_teste1();

//   Declaração das entradas
    reg CLKBasys = 0;
    reg BtnReset = 0, BtnStart = 0;

//   Declaração das saídas
    wire [6:0] Seg;
    wire DP;
    wire [3:0] An;
    wire LedReady, LedBeat, LedWaiting;
    wire [5:0] LedEstado;

//   Instanciação da UUT com tempos curtos para a simulação
    GuitarHero #(.METADE(2), .ESTAVEL(4), .INICIAL(40), .PASSO(4), .MINIMO(20)) UUT (
        .CLKBasys(CLKBasys),
        .BtnReset(BtnReset),
        .BtnStart(BtnStart),
        .Seg(Seg),
        .DP(DP),
        .An(An),
        .LedReady(LedReady),
        .LedBeat(LedBeat),
        .LedWaiting(LedWaiting),
        .LedEstado(LedEstado)
    );

//   Relógio de 100 MHz (período de 10ns)
    always #5 CLKBasys = ~CLKBasys;

//   Estados one-hot: LedEstado = {S5, S4, S3, S2, S1, S0}
    localparam S0 = 6'b000001, S1 = 6'b000010, S2 = 6'b000100, S4 = 6'b010000;

    integer erros = 0, verificacoes = 0;
    task erro;
```

```
43          begin
44              $display("ERRO t=%0t: Estado=%b LEDs=%b%b%b Pos=%b %b %b %b Periodo=%0d Seg=%b",
45                       $time, LedEstado, LedReady, LedWaiting, LedBeat,
46                       UUT.Pos0, UUT.Pos1, UUT.Pos2, UUT.Pos3, UUT.temporizador.Periodo, Seg);
47              erros = erros + 1;
48          end
49      endtask
50
51  //  Confere o estado e os LEDs Ready, Waiting e Beat
52      task confere_leds(input [5:0] estado, input ready, input waiting, input beat);
53          begin
54              verificacoes = verificacoes + 1;
55              if ({LedEstado, LedReady, LedWaiting, LedBeat} !== {estado, ready, waiting, beat}) erro;
56          end
57      endtask
58
59  //  Confere as cordas da memória (Pos0 no display da esquerda ... Pos3 no da direita)
60      task confere_cordas(input [1:0] p0, input [1:0] p1, input [1:0] p2, input [1:0] p3);
61          begin
62              verificacoes = verificacoes + 1;
63              if ({UUT.Pos0, UUT.Pos1, UUT.Pos2, UUT.Pos3} !== {p0, p1, p2, p3}) erro;
64          end
65      endtask
66
67  //  Confere o período do temporizador (em ciclos)
68      task confere_periodo(input integer periodo);
69          begin
70              verificacoes = verificacoes + 1;
71              if (UUT.temporizador.Periodo !== periodo) erro;
72          end
73      endtask
74
75  //  Espera pelo passo seguinte do jogo: entrada em S4, quando a memória já deslocou
76      task passo;
77          begin
78              @(posedge LedEstado[4]);
79              #2;
80          end
81      endtask
82
83      initial begin
84          // Depois de configurar a FPGA o Reset é libertado (2 flancos) e a máquina passa de S0 a S1
85          #200 confere_leds(S1, 1, 0, 0);                  // à espera de Start, Ready aceso
86               confere_cordas(2'b00, 2'b00, 2'b00, 2'b00); // displays apagados
87               confere_periodo(40);
88
89          // Carregar Start com um ressalto (mais curto do que o anti-ressalto)
90          BtnStart = 1; #10 BtnStart = 0; #10 BtnStart = 1;
91          #200 confere_leds(S2, 1, 0, 0);                  // S2 enquanto Start está carregada
92          BtnStart = 0;                                    // largar Start: o jogo começa
93
94          // Passos do jogo: entra uma corda nova em Pos0, as outras avançam, o Beat troca
95          // e o período diminui 4 ciclos (o S5 do passo anterior) até ao mínimo de 20
96          passo; confere_leds(S4, 0, 1, 1); confere_cordas(2'b10, 2'b00, 2'b00, 2'b00); confere_periodo(40);
97          @(negedge An[3]) #20;                            // display da esquerda ativo
98          verificacoes = verificacoes + 1;
99          if (Seg !== 7'b1111110) erro;                    // Pos0 = 10: corda do meio (segmento g)
100         passo; confere_leds(S4, 0, 1, 0); confere_cordas(2'b00, 2'b10, 2'b00, 2'b00); confere_periodo(36);
101         passo; confere_leds(S4, 0, 1, 1); confere_cordas(2'b11, 2'b00, 2'b10, 2'b00); confere_periodo(32);
102         passo; confere_leds(S4, 0, 1, 0); confere_cordas(2'b00, 2'b11, 2'b00, 2'b10); confere_periodo(28);
103         passo; confere_leds(S4, 0, 1, 1); confere_cordas(2'b01, 2'b00, 2'b11, 2'b00); confere_periodo(24);
104         passo; confere_leds(S4, 0, 1, 0); confere_cordas(2'b11, 2'b01, 2'b00, 2'b11); confere_periodo(20);
```

```
105          passo; confere_leds(S4, 0, 1, 1); confere_cordas(2'b10, 2'b11, 2'b01, 2'b00); confere_periodo(20);
106
107          // Reset a meio do jogo
108          #200 BtnReset = 1;
109          #1   confere_leds(S0, 0, 0, 0);                 // S0 de imediato (sem flanco); Beat apagado
110               confere_cordas(2'b00, 2'b00, 2'b00, 2'b00); // memória limpa (CLR assíncrono)
111          #50  BtnReset = 0;
112          #100 confere_leds(S1, 1, 0, 0);                 // de volta a S1, à espera de Start
113               confere_periodo(40);                       // período inicial reposto
114
115          if (erros == 0) $display("Simulacao OK (%0d verificacoes)", verificacoes);
116          else            $display("Simulacao FALHOU (%0d erros em %0d verificacoes)", erros, verificacoes);
117          $finish;
118      end
119
120  //   Se o jogo não avançar, a simulação acaba aqui
121      initial begin
122          #10000 $display("Simulacao FALHOU (tempo limite: o jogo nao avancou)");
123          $finish;
124      end
125
126  endmodule
```